

Software Testing Guide Book

Part I: Fundamentals of Software Testing

Ajitha, Amrish Shah, Ashna Datye, Bharathy J, Deepa M G, James M, Jayapradeep J, Jeffin Jacob M, Kapil Mohan Sharma, Leena Warriar, Mahesh, Michael Frank, Muhammad Kashif Jamil Narendra N, Naveed M, Phaneendra Y, Prathima N, Ravi Kiran N, Rajeev D, Sarah Salahuddin, Siva Prasad B, Shalini R, Shilpa D, Subramanian D Ramprasad, Sunitha C N, Sunil Kumar M K, Usha Padmini K, Winston George and Harinath P V

Copyright (c) SofTReL 2004. Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.2 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts. A copy of the license is included in the section entitled "GNU Free Documentation License".

Software Testing Research Lab

<http://www.SofTReL.org>

Revision History

Ver. No.	Date	Description	Author
0.1	06-Apr-04	Initial document creation	Harinath, on behalf of STGB Team.
0.2	01-May-04	Incorporated Review Comments	Harinath, on behalf of STGB Team.
0.3	03-July-04	Draft Release	Harinath, on behalf of STGB Team

Table of Contents

Software Testing Guide Book.....	1
1. The Software Testing Guide Book.....	6
Forward.....	6
About SofTReL.....	7
Purpose of this Document.....	7
Authors.....	8
Intended Audience.....	9
How to use this Document.....	9
What this Guide Book is not.....	9
How to Contribute.....	9
Future Enhancements.....	9
Copyrights.....	9
2. What is Software Testing and Why is it Important?.....	10
3. Types of Development Systems.....	12
3.1 Traditional Development Systems.....	12
3.2 Iterative Development.....	12
3.3 Maintenance System.....	12
3.4 Purchased/Contracted Software.....	13
4. Types of Software Systems.....	13
4.1 Batch Systems.....	13
4.2 Event Control Systems.....	13
4.3 Process Control Systems.....	13
4.4 Procedure Control Systems.....	14
4.5 Advanced Mathematical Models.....	14
4.6 Message Processing Systems.....	14
4.7 Diagnostic Software Systems.....	14
4.8 Sensor and Signal Processing Systems.....	14
4.9 Simulation Systems.....	15
4.10 Database Management Systems.....	19
4.11 Data Acquisition	19
4.12 Data Presentation	19
4.13 Decision and Planning Systems.....	19
4.14 Pattern and Image Processing Systems.....	19
4.15 Computer System Software Systems.....	20
4.16 Software Development Tools.....	20
5. Heuristics of Software Testing.....	20
Testability.....	20
Visibility.....	20
Control.....	21
A BROADER VIEW.....	22
6. When Testing should occur?.....	24
7. The Test Development Life Cycle (TDLC).....	28
8. When should Testing stop?.....	31
http://www.SofTReL.org	

9. Verification Strategies.....	31
9.1 Review.....	31
Technical Reviews	32
Who is involved in Requirement Review?.....	33
Input Criteria.....	33
Exit Criteria.....	33
What is Design Review?.....	33
Who involve in Design Review?.....	34
Input Criteria.....	34
Exit Criteria.....	34
What is Code Review?.....	34
Who is involved in Code Review?.....	34
Input Criteria.....	34
Exit Criteria.....	34
9.2 Walkthrough.....	34
9.3 Inspection.....	36
10. Testing Types and Techniques.....	37
10.1 White Box Testing.....	39
10.1.1 Basis Path Testing.....	43
10.1.2 Flow Graph Notation.....	43
10.1.3 Cyclomatic Complexity.....	43
10.1.4 Graph Matrices.....	43
10.1.5 Control Structure Testing.....	44
10.1.6 Loop Testing.....	44
10.2 Black Box Testing	45
Advantages of Black Box Testing.....	45
10.2.1 Graph Based Testing Methods.....	46
10.2.2 Error Guessing.....	46
10.2.3 Boundary Value Analysis.....	46
10.2.4 Equivalence Partitioning.....	48
10.2.5 Comparison Testing.....	48
10.2.6 Orthogonal Array Testing.....	48
11. Designing Test Cases.....	48
12. Validation Phase.....	49
12.1 Unit Testing.....	49
The programmer implements some functionality for the system to be developed.	
The same is tested by referring the unit test cases. While testing that functionality	
if any defects have been found, they are recorded using the defect logging tool	
whichever is applicable. The programmer fixes the bugs found and tests the same	
for any errors.....	49
Stubs and Drivers.....	49
12.2 Integration Testing.....	54
12.2.1 Top-Down Integration.....	54
12.2.2 Bottom-Up Integration.....	54
12.3 System Testing.....	55
12.3.1 Compatibility Testing.....	55

12.3.2 Recovery Testing.....	56
12.3.3 Usability Testing.....	56
Performing the test.....	57
tools available for Usability Testing.....	57
End goals of Usability Testing.....	59
12.3.4 Security Testing.....	59
12.3.5 Stress Testing.....	59
12.3.6 Performance Testing	60
Response Time.....	60
Bug Fixing.....	69
12.3.7 Content Management Testing.....	69
12.3.8 Regression Testing	69
12.4 Alpha Testing.....	72
12.5 User Acceptance Testing.....	73
12.6 Installation Testing.....	73
12.7 Beta Testing	74
13. Understanding Exploratory Testing.....	74
What is formalized Exploratory Testing?.....	76
Defect Driven Exploratory Testing:.....	86
Pros and Cons:.....	87
Pros.....	87
14. Understanding Scenario Based Testing.....	91
15. Understanding Agile Testing.....	92
Agile Development Methodologies.....	93
Test Interaction Model.....	96
16. API Testing.....	97
Behavioral pre-setters.....	101
17. Understanding Rapid Testing.....	104
18. Test Ware Development	106
18.1 Test Strategy.....	106
18.2 Test Plan.....	109
18.3 Test Case Documents.....	115
Characteristics of Good Scenarios.....	116
System Test Cases: -.....	120
19. Defect Management.....	121
19.1 What is a Defect?.....	121
For a test engineer, a defect is following: -	121
19.2 Defect Taxonomies.....	122
19.3 Life Cycle of a Defect.....	123
20. Metrics for Testing.....	123
Goal Purpose Issue Object View Point.....	137
References.....	138
GNU Free Documentation License.....	139

1. The Software Testing Guide Book

Forward

Software Testing has gained a phenomenal importance in the recent years in the System Development Life Cycle. Many learned people have worked on the topic and provided various techniques and methodologies for effective and efficient testing. Today, even though we have many books and articles on Software Test Engineering, many people are fallacious in understanding the underlying concepts of the subject.

Software Testing Guide Book (STGB) is an open source project aimed at bringing the technicalities of Software Testing into one place and arriving at a common understanding.

This guide book has been authored by professionals who have been exposed to Testing various applications. We wanted to bring out a base knowledge bank where Testing enthusiasts can start to learn the science and art of Software Testing, and this is how this book has come out.

This guide book does not provide any specific methodologies to be followed for Testing, instead provides a conceptual understanding of the same.

Note to the Reader:

It is not our intention to tell you that this is a one-stop place for learning Testing. This is just a guide. Many eminent scientists have researched every topic you find in this book. We have just compiled everything in one place and made sure we explained each topic relating it to the practical world as we experienced it. If you find any subject matter that might look like we have copied from any existing book, we request you to let us know. It is not our intention to copy any material, and we brought out this book just to help Testing aspirants to have a basic understanding of the subject and guide them to be good at their job. All the material in this document is written in plain English, as we understand testing.

Please send in your comments, suggestions or a word of encouragement to the [team](#).

Regards,
The SofTReL Team

About SofTReL

The Software Testing Research Lab (SofTReL) is a non-profit organization dedicated for Research and Advancements of Software Testing.

The concept of having a common place for Software Testing Research was formulated in 2001. Initially we named it ‘Software Quality and Engineering’. Recently in March 2004, we renamed it to “Software Testing Research Lab” – SofTReL.

Professionals who are currently working with the industry and possess rich experience in testing form the members of the Lab.

Visit <http://www.softrel.org> for more information.

Purpose of this Document

This document does not provide the reader with short cut’s to perform testing in daily life, but instead explains the various methodologies and techniques which have been proposed by eminent scientists in an easy and understandable way.

This guide book is divided into three parts:

Part I – Fundamentals of Software Testing

This section addresses the fundamentals of Software Testing and their practical application in real life.

Part II – Software Testing for various Architectures

This section would concentrate in explaining testing applications under various architectures like Client/Server, Web, Pocket PC, Mobile and Embedded.

Part III – Platform Specific Testing

This section addresses testing C++ and Java applications using white box testing methodologies.

This is Part I. All updates on the project are available at

<http://www.SofTReL.org/stgb.html>.

Authors

The guide book has been authored by professionals who 'Test' everyday.

Ajitha - GrayLogic Corporation, New Jersey, USA

Amrisha Shah - MAQSoftware, Mumbai

Ashna Datye - RS Tech Inc, Canada

Bharathy Jayaraman - Ivesia Solutions (I) Pvt Limited, Chennai

Deepa M G - Ocwen Technology Xchange, Bangalore

James M - CSS, Chennai

Jayapradeep Jiothis - Satyam Computer Services, Hyderabad

Jeffin Jacob Mathew - ICFAI Business School, Hyderabad

Kapil Mohan Sharma - Pixtel Communitations, New Delhi

Leena Warriar - Wipro Technologies, Bangalore

Mahesh, iPointSoft, Hyderabad

Michael Frank - USA

Muhammad Kashif Jamil, Avanza Solutions, Karachi, Pakistan

Narendra Nagaram - Satyam Computer Services, Hyderabad

Naveed Mohammad - vMoksha, Bangalore

Phaneendra Y - Wipro Technologies, Bangalore

Prathima Nagaprakash - Wipro Technologies, Bangalore

Ravi Kiran N - Andale, Bangalore

Rajeev Daithankar - Persistent Systems Pvt. Ltd., Pune

Sarah Salahuddin - Arc Solutions, Pakistan

Siva Prasad Badimi - Danlaw Technologies, Hyderabad

Shalini Ravikumar - USA

Shilpa Dodla - Decatrend Technologies, Chennai

Subramanian Dattaramprasad - MindTeck, Bangalore

Sunitha C N - Infosys Technologies, Mysore

Sunil Kumar M K - Yahoo! India, Bangalore

Usha Padmini Kandala - Virtusa Corp, Massachusetts

Winston George - VbiZap Soft Solutions (P) Ltd., Chennai

Harinath - SofTReL, Bangalore

Intended Audience

This guide book is aimed at all Testing Professionals – from a beginner to an advanced user. This book would provide a baseline understanding of the conceptual theory.

How to use this Document

This book can be used as a guide for performing the Testing activities. A ‘guide’ here, we mean that this can provide you a road map as to how to approach a specific problem with respect to Testing.

What this Guide Book is not

This guide book is definitely not a silver/gold/diamond bullet which can help you to test any application. Instead this book would be reference help to perform Testing.

How to Contribute

This is an open source project. If you are interested in contributing to the book or to the Lab, please do write in to [stgb at SoftReL dot org](mailto:stgb@SoftReL.org). We need your expertise in the research activities.

Future Enhancements

This is the first part of the three-part Software Testing Guide Book (STGB) series. You can visit <http://www.softrel.org/stgb.html> for updates on the Project.

Copyrights

SoftReL is not proposing the Testing methodologies, types and various other concepts. We tried presenting each and every theoretical concept of Software Testing with a live example for easier understanding of the subject and arriving at a common understanding of Software Test Engineering.

However, we did put in few of our proposed ways to achieve specific tasks and these are governed by The GNU Free Documentation License (GNU-FDL). Please visit <http://www.gnu.org/doc/doc.html> for complete guidelines of the license or alternatively you can find the license towards the end of this document

2. What is Software Testing and Why is it Important?

A brief history of Software engineering and the SDLC.

The software industry has evolved through 4 eras, 50's –60's, mid 60's –late 70's, mid 70's- mid 80's, and mid 80's-present. Each era has its own distinctive characteristics, but over the years the software's have increased in size and complexity. Several problems are common to almost all of the eras and are discussed below.

The Software Crisis dates back to the 1960's when the primary reasons for this situation were less than acceptable software engineering practices. In the early stages of software there was a lot of interest in computers, a lot of code written but no established standards. Then in early 70's a lot of computer programs started failing and people lost confidence and thus an industry crisis was declared. Various reasons leading to the crisis included:

- ☐ Hardware advances outpacing the ability to build software for this hardware.
- ☐ The ability to build in pace with the demands.
- ☐ Increasing dependency on software's
- ☐ Struggle to build reliable and high quality software
- ☐ Poor design and inadequate resources.

This crisis though identified in the early years, exists to date and we have examples of software failures around the world. Software is basically considered a failure if the project is terminated because of costs or overrun schedules, if the project has experienced overruns in excess of 50% of the original or if the software results in client lawsuits. Some *examples of failures* include failure of Air traffic control systems, failure of medical software, and failure in telecommunication software. The primary reason for these failures other than those mentioned above is due to bad software engineering practices adopted. Some of the worst software practices include:

- ☐ No historical software-measurement data.
- ☐ Rejection of accurate cost estimates.
- ☐ Failure to use automated estimating and planning tools.
- ☐ Excessive, irrational schedule pressure and creep in user requirements.
- ☐ Failure to monitor progress and to perform risk management.
- ☐ Failure to use design reviews and code inspections.

To avoid these failures and thus improve the record, what is needed is a better understanding of the process, better estimation techniques for cost time and quality

measures. But the question is, what is a process? Process transform inputs to outputs i.e. a product. A software process is a set of activities, methods and practices involving transformation that people use to develop and maintain software.

At present a large number of problems exist due to a chaotic software process and the occasional success depends on individual efforts. Therefore to be able to deliver successful software projects, a focus on the process is essential since a focus on the product alone is likely to miss the scalability issues, and improvements in the existing system. This focus would help in the predictability of outcomes, project trends, and project characteristics.

The process that has been defined and adopted needs to be managed well and thus process management comes into play. *Process management* is concerned with the knowledge and management of the software process, its technical aspects and also ensures that the processes are being followed as expected and improvements are shown.

From this we conclude that a set of defined processes can possibly save us from software project failures. But it is nonetheless important to note that the process alone cannot help us avoid all the problems, because with varying circumstances the need varies and the process has to be adaptive to these varying needs. Importance needs to be given to the human aspect of software development since that alone can have a lot of impact on the results, and effective cost and time estimations may go totally waste if the human resources are not planned and managed effectively. Secondly, the reasons mentioned related to the software engineering principles may be resolved when the needs are correctly identified. Correct identification would then make it easier to identify the best practices that can be applied because one process that might be suitable for one organization may not be most suitable for another.

Therefore to make a successful product a combination of Process and Technicalities will be required under the umbrella of a well-defined process.

Having talked about the Software process overall, it is important to identify and relate the role software testing plays not only in producing quality software but also maneuvering the overall process.

The computer society defines testing as follows: “**Testing** -- A verification method that applies a controlled set of conditions and stimuli for the purpose of finding errors. This is the most desirable method of verifying the functional and performance requirements.

Test results are documented proof that requirements were met and can be repeated. The resulting data can be reviewed by all concerned for confirmation of capabilities.” There may be many definitions of software testing and many which appeal to us from time to time, but its best to start by defining testing and then move on depending on the requirements or needs.

3. Types of Development Systems

The type of development project refers to the environment/methodology in which the software will be developed. Different testing approaches need to be used for different types of projects, just as different development approaches.

3.1 Traditional Development Systems

The Traditional Development System has the following characteristics:

- The traditional development system uses a system development methodology.
- The user knows what the customer requires (Requirements are clear from the customer).
- The development system determines the structure of the application.

What do you do while testing:

- Testing happens at the end of each phase of development.
- Testing should concentrate if the requirements match the development.
- Functional testing is required.

3.2 Iterative Development

During the Iterative Development:

- The requirements are not clear from the user (customer).
- The structure of the software is pre-determined.

Testing of Iterative Development projects should concentrate only if the CASE (Computer Aided Software Engineering) tools are properly utilized and the functionality is thoroughly tested.

3.3 Maintenance System

The Maintenance System is where the structure of the program undergoes changes. The system is developed and being used, but it demands changes in the functional aspects of the system due to various reasons.

Testing Maintenance Systems requires structural testing. Top priority should be put into Regression Testing.

3.4 Purchased/Contracted Software

At times it may be required that you purchase software to integrate with your product or outsource the development of certain components of your product. This is Purchased or Contracted Software.

When you need to integrate third party software to your existing software, this demands the testing of the purchased software with your requirements. Since the two systems are designed and developed differently, the integration takes the top priority during testing. Also, Regression Testing of the integrated software is a must to cross check if the two software's are working as per the requirements.

4. Types of Software Systems

The type of software system refers to the processing that will be performed by that system. This contains the following software system types.

4.1 Batch Systems

The Batch Systems are a set of programs that perform certain activities which do not require any input from the user.

A practical example is that when you are typing something on a word document, you press the key you require and the same is printed on the monitor. But processing (converting) the user input of the key to the machine understandable language, making the system understand what to be displayed and in return the word document displaying what you have typed is performed by the batch systems. These batch systems contain one or more Application Programming Interface (API) which perform various tasks.

4.2 Event Control Systems

Event Control Systems process real time data to provide the user with results for what command (s) he is given.

For example, when you type on the word document and press Ctrl + S, this tells the computer to save the document. How this is performed instantaneously? These real time command communications to the computer are provided by the Event Controls that are pre-defined in the system.

4.3 Process Control Systems

There are two or more different systems that communicate to provide the end user a specific utility. When two systems communicate, the co-ordination or data transfer becomes vital. Process Control Systems are the one's which receive data from a different

system and instructs the system which sent the data to perform specific tasks based on the reply sent by the system which received the data.

4.4 Procedure Control Systems

Procedure Control Systems are the one's which control the functions of another system.

4.5 Advanced Mathematical Models

Systems, which make use of heavy mathematics, fall into the category of Mathematical Models. Usually all the computer software make use of mathematics in some way or the other. But, Advance Mathematical Models can be classified when there is heavy utilization of mathematics for performing certain actions. An example of Advanced Mathematical Model can be simulation systems which uses graphics and controls the positioning of software on the monitor or Decision and Strategy making software's.

4.6 Message Processing Systems

A simple example is the SMS management software used by Mobile operator's which handle incoming and outgoing messages. Another system, which is noteworthy is the system used by paging companies.

4.7 Diagnostic Software Systems

The Diagnostic Software System is one that helps in diagnosing the computer hardware components.

When you plug in a new device to your computer and start it, you can see the diagnostic software system doing some work. The "New Hardware Found" dialogue can be seen as a result of this system. Today, almost all the Operating System's come packed with Diagnostic Software Systems.

4.8 Sensor and Signal Processing Systems

The message processing systems help in sending and receiving messages. The Sensor and Signal Processing Systems are more complex because these systems make use of mathematics for signal processing. In a signal processing system the computer receives input in the form of signals and then transforms the signals to a user understandable output.

4.9 Simulation Systems

A simulation system is a software application, some times used in combination with specialized hardware, which re-creates or simulates the complex behavior of a system in its real environment. It can be defined in many ways:

"The process of designing a model of a real system and conducting experiments with this model for the purpose of understanding the behavior of the system and/or evaluating various strategies for the operation of the system"-- Introduction to Simulation Using SIMAN, by C. D. Pegden, R. E. Shannon and R. P. Sadowski, McGraw-Hill, 1990.

"A simulation is a software package (sometimes bundled with special hardware input devices) that re-creates or simulates, albeit in a simplified manner, a complex phenomena, environment, or experience, providing the user with the opportunity for some new level of understanding. It is interactive, and usually grounded in some objective reality. A simulation is based on some underlying computational model of the phenomena, environment, or experience that it is simulating. (In fact, some authors use model and modeling as synonyms of simulation.)" --Kurt Schumaker, A Taxonomy of Simulation Software." Learning Technology Review.

In simple words simulation is nothing but a representation of a real system. In a programmable environment, simulations are used to study system behavior or test the system in an artificial environment that provides a limited representation of the real environment.

Why Simulation Systems

Simulation systems are easier, cheaper, and safer to use than real systems, and often the only way to build the real systems. For example, learning to fly a fighter plane using a simulator is much safer and less expensive than learning on a real fighter plane. System simulation mimics the operation of a real system such as the operation in a bank, or the running of the assembly line in a factory etc.

Simulation in the early stage of design cycle is important because the cost of mistakes increases dramatically later in the product life cycle. Also, simulation software can analyze the operation of a real system without the involvement of an expert, i.e. it can also be analyzed with a non-expert like a manager.

How to Build Simulation Systems

In order to create a simulation system we need a realistic model of the system behavior. One way of simulation is to create smaller versions of the real system.

<http://www.SofTReL.org>

The simulation system may use only software or a combination of software and hardware to model the real system. The simulation software often involves the integration of artificial intelligence and other modeling techniques.

What applications fall under this category?

Simulation is widely used in many fields. Some of the applications are:

- Models of planes and cars that are tested in wind tunnels to determine the aerodynamic properties.
- Used in computer Games (E.g. SimCity, car games etc). This simulates the working in a city, the roads, people talking, playing games etc.
- War tactics that are simulated using simulated battlefields.
- Most Embedded Systems are developed by simulation software before they ever make it to the chip fabrication labs.
- Stochastic simulation models are often used to model applications such as weather forecasting systems.
- Social simulation is used to model socio-economic situations.
- It is extensively used in the field of operations research.

What are the Characteristics of Simulation Systems?

Simulation Systems can be characterized in numerous ways depending on the characterization criteria applied. Some of them are listed below.

Deterministic Simulation Systems

Deterministic Simulation Systems have completely predictable outcomes. That is, given a certain input we can predict the exact outcome. Another feature of these systems is idempotency, which means that the results for any given input are always the same.

Examples include population prediction models, atmospheric science etc.

Stochastic Simulation Systems

Stochastic Simulation systems have models with random variables. This means that the exact outcome is not predictable for any given input, resulting in potentially very different outcomes for the same input.

Static Simulation Systems

Static Simulation systems use statistical models in which time does not play any role. These models include various probabilistic scenarios which are used to calculate the results of any given input. Examples of such systems include financial portfolio valuation models. The most common simulation technique used in these models is the Monte Carlo Simulation.

Dynamic Simulation Systems

A dynamic simulation system has a model that accommodates for changes in data over time. This means that the input data affecting the results will be entered into the simulation during its entire lifetime than just at the beginning. A simulation system used to predict the growth of the economy may need to incorporate changes in economic data, is a good example of a dynamic simulation system.

Discrete Simulation Systems

Discrete Simulation Systems use models that have discrete entities with multiple attributes. Each of these entities can be in any state, at any given time, represented by the values of its attributes. . The state of the system is a set of all the states of all its entities.

This state changes one discrete step at a time as events happens in the system. Therefore, the actual designing of the simulation involves making choices about which entities to model, what attributes represent the Entity State, what events to model, how these events impact the entity attributes, and the sequence of the events. Examples of these systems are simulated battlefield scenarios, highway traffic control systems, multiteller systems, computer networks etc.

Continuous Simulation Systems

If instead of using a model with discrete entities we use data with continuous values, we will end up with continuous simulation. For example instead of trying to simulate battlefield scenarios by using discrete entities such as soldiers and tanks, we can try to model behavior and movements of troops by using differential equations.

Social Simulation Systems

Social simulation is not a technique by itself but uses the various types of simulation described above. However, because of the specialized application of those techniques for social simulation it deserves a special mention of its own.

The field of social simulation involves using simulation to learn about and predict various social phenomenon such as voting patterns, migration patterns, economic decisions made by the general population, etc. One interesting application of social simulation is in a field called artificial life which is used to obtain useful insights into the formation and evolution of life.

What can be the possible test approach?

A simulation system's primary responsibility is to replicate the behavior of the real system as accurately as possible. Therefore, a good place to start creating a test plan would be to understand the behavior of the real system.

<http://www.SofTReL.org>

Subjective Testing

Subjective testing mainly depends on an expert's opinion. An expert is a person who is proficient and experienced in the system under test. Conducting the test involves test runs of the simulation by the expert and then the expert evaluates and validates the results based on some criteria.

One advantage of this approach over objective testing is that it can test those conditions which cannot be tested objectively. For example, an expert can determine whether the joystick handling of the flight feels "right".

One disadvantage is that the evaluation of the system is based on the "expert's" opinion, which may differ from expert to expert. Also, if the system is very large then it is bound to have many experts. Each expert may view it differently and can give conflicting opinions. This makes it difficult to determine the validity of the system. Despite all these disadvantages, subjective testing is necessary for testing systems with human interaction.

Objective Testing

Objective testing is mainly used in systems where the data can be recorded while the simulation is running. This testing technique relies on the application of statistical and automated methods to the data collected.

Statistical methods are used to provide an insight into the accuracy of the simulation. These methods include hypothesis testing, data plots, principle component analysis and cluster analysis.

Automated testing requires a knowledge base of valid outcomes for various runs of simulation. This knowledge base is created by domain experts of the simulation system being tested. The data collected in various test runs is compared against this knowledge base to automatically validate the system under test. An advantage of this kind of testing is that the system can continually be regression tested as it is being developed.

Statistical Methods

Statistical methods are used to provide an insight into the accuracy of the simulation. These methods include hypothesis testing, data plots, principle component analysis and cluster analysis.

Automated Testing

Automated testing requires a knowledge base of valid outcomes for various runs of simulation. This knowledge base is created by domain experts of the simulation system being tested. The data collected in various test runs is compared against this knowledge

base to automatically validate the system under test. An advantage of this kind of testing is that the system can continually be regression tested as it is being developed.

4.10 Database Management Systems

As the name denotes, the Database Management Systems (DBMS) handles the management of databases. It is basically a collection of programs that enable the storage, modification and extraction from the Database. The DBMS, as they are often referred to as, can be of various different types ranging from small systems that run on PC's to Mainframe's. The following can be categorized as example of DBMS:

- Computerized Library Systems.
- Automated Teller Machines.
- Passenger Reservation Systems.
- Inventory Systems.

4.11 Data Acquisition

Data Acquisition systems, taken in real time data and store them for future use. A simple example of Data Acquisition system can be a ATC (Air Traffic Control) Software which takes in real time data of the position and speed of the flight and stores it in compressed form for later use.

4.12 Data Presentation

Data Presentation software stores data and displays the same to the user when required. An example is a Content Management System. You have a web site and this is in English, you also have your web site in other languages. The user can select the language he wishes to see and the system displays the same web site in the user chosen language. You develop your web site in various languages and store them on the system. The system displays the required language, the user chooses.

4.13 Decision and Planning Systems

These systems use Artificial Intelligence techniques to provide decision-making solutions to the user.

4.14 Pattern and Image Processing Systems

These systems are used for scanning, storing, modifying and displaying graphic images. The use of such systems is now being increased as research tests are being conducted in visual modeling and their use in our daily lives is increasing. These systems are used

for security requests such as diagnosing photograph, thumb impression of the visitor etc.

4.15 Computer System Software Systems

These are the normal computer software's, that can be used for various purposes.

4.16 Software Development Tools

These systems ease the process of Software Development.

5. Heuristics of Software Testing

Testability

Software testability is how easily, completely and conveniently a computer program can be tested.

Software engineers design a computer product, system or program keeping in mind the product testability. Good programmers are willing to do things that will help the testing process and a checklist of possible design points, features and so on can be useful in negotiating with them.

Here are the two main heuristics of software testing.

1. Visibility
2. Control

Visibility

Visibility is our ability to observe the states and outputs of the software under test. Features to improve the visibility are

- Access to Code

Developers must provide full access (source code, infrastructure, etc) to testers. The Code, change records and design documents should be provided to the testing team. The testing team should read and understand the code.

- Event logging

The events to log include User events, System milestones, Error handling and completed transactions. The logs may be stored in files, ring buffers in memory, and/or serial ports. Things to be logged include description of event, timestamp, subsystem, resource usage and severity of event. Logging should be adjusted by subsystem and type. Log file report internal errors, help in isolating defects, and give useful information about context, tests, customer usage and test coverage.

The more readable the Log Reports are, the easier it becomes to identify the defect cause and work towards corrective measures.

- Error detection mechanisms

Data integrity checking and System level error detection (e.g. Microsoft Appviewer) are useful here. In addition, Assertions and probes with the following features are really helpful

- Code is added to detect internal errors.
- Assertions abort on error.
- Probes log errors.
- Design by Contract theory---This technique requires that assertions be defined for functions. Preconditions apply to input and violations implicate calling functions while post-conditions apply to outputs and violations implicate called functions. This effectively solves the oracle problem for testing.

- Resource Monitoring

Memory usage should be monitored to find memory leaks. States of running methods, threads or processes should be watched (Profiling interfaces may be used for this.). In addition, the configuration values should be dumped. Resource monitoring is of particular concern in applications where the load on the application in real time is estimated to be considerable.

Control

Control refers to our ability to provide inputs and reach states in the software under test.

The features to improve controllability are:

- Test Points

Allow data to be inspected, inserted or modified at points in the software. It is especially useful for dataflow applications. In addition, a pipe and filters architecture provides many opportunities for test points.

- Custom User Interface controls

Custom UI controls often raise serious testability problems with GUI test drivers. Ensuring testability usually requires:

- Adding methods to report necessary information
- Customizing test tools to make use of these methods

- Getting a tool expert to advise developers on testability and to build the required support.
 - Asking third party control vendors regarding support by test tools.
- Test Interfaces

Interfaces may be provided specifically for testing e.g. Excel and Xconq etc.

Existing interfaces may be able to support significant testing e.g. InstallSheild, AutoCAD, Tivoli, etc.
 - Fault injection

Error seeding---instrumenting low level I/O code to simulate errors---makes it much easier to test error handling. It can be handled at both system and application level, Tivoli, etc.
 - Installation and setup

Testers should be notified when installation has completed successfully. They should be able to verify installation, programmatically create sample records and run multiple clients, daemons or servers on a single machine.

A BROADER VIEW

Below are given a broader set of characteristics (usually known as James Bach heuristics) that lead to testable software.

Categories of Heuristics of software testing

- Operability

The better it works, the more efficiently it can be tested.

The system should have few bugs, no bugs should block the execution of tests and the product should evolve in functional stages (simultaneous development and testing).
- Observability

What we see is what we test.

 - Distinct output should be generated for each input
 - Current and past system states and variables should be visible during testing
 - All factors affecting the output should be visible.

- Incorrect output should be easily identified.
- Source code should be easily accessible.
- Internal errors should be automatically detected (through self-testing mechanisms) and reported.
- Controllability

The better we control the software, the more the testing process can be automated and optimized.

Check that

 - All outputs can be generated and code can be executed through some combination of input.
 - Software and hardware states can be controlled directly by the test engineer.
 - Inputs and output formats are consistent and structured.
 - Test can be conveniently, specified, automated and reproduced.
- Decomposability

By controlling the scope of testing, we can quickly isolate problems and perform effective and efficient testing.

The software system should be built from independent modules which can be tested independently.
- Simplicity

The less there is to test, the more quickly we can test it.

The points to consider in this regard are functional (e.g. minimum set of features), structural (e.g. architecture is modularized) and code (e.g. a coding standard is adopted) simplicity.
- Stability

The fewer the changes, the fewer are the disruptions to testing.

The changes to software should be infrequent, controlled and not invalidating existing tests. The software should be able to recover well from failures.
- Understandability

The more information we will have, the smarter we will test.

The testers should be able to understand well the design, changes to the design and the dependencies between internal, external and shared components.

Technical documentation should be instantly accessible, accurate, well organized, specific and detailed.
- Suitability

The more we know about the intended use of the software, the better we can organize our testing to find important bugs.

The above heuristics can be used by a software engineer to develop a software configuration (i.e. program, data and documentation) that is convenient to test and verify.

6. When Testing should occur?

Wrong Assumption

Testing is sometimes incorrectly thought as an after-the-fact activity; performed after programming is done for a product. Instead, testing should be performed at every development stage of the product. Test data sets must be derived and their correctness and consistency should be monitored throughout the development process. If we divide the lifecycle of software development into “Requirements Analysis”, “Design”, “Programming/Construction” and “Operation and Maintenance”, then testing should accompany each of the above phases. If testing is isolated as a single phase late in the cycle, errors in the problem statement or design may incur exorbitant costs. Not only must the original error be corrected, but the entire structure built upon it must also be changed. Therefore, testing should not be isolated as an inspection activity. Rather testing should be involved throughout the SDLC in order to bring out a quality product.

Testing Activities in Each Phase

The following testing activities should be performed during the phases

- Requirements Analysis - (1) Determine correctness (2) Generate functional test data.
- Design - (1) Determine correctness and consistency (2) Generate structural and functional test data.
- Programming/Construction - (1) Determine correctness and consistency (2) Generate structural and functional test data (3) Apply test data (4) Refine test data.
- Operation and Maintenance - (1) Retest.

Now we consider these in detail.

Requirements Analysis

<http://www.SofTReL.org>

The following test activities should be performed during this stage.

- *Invest in analysis at the beginning of the project* - Having a clear, concise and formal statement of the requirements facilitates programming, communication, error analysis and test data generation.

The requirements statement should record the following information and decisions:

1. Program function - What the program must do?
2. The form, format, data types and units for input.
3. The form, format, data types and units for output.
4. How exceptions, errors and deviations are to be handled.
5. For scientific computations, the numerical method or at least the required accuracy of the solution.
6. The hardware/software environment required or assumed (e.g. the machine, the operating system, and the implementation language).

Deciding the above issues is one of the activities related to testing that should be performed during this stage.

- *Start developing the test set at the requirements analysis phase* - Data should be generated that can be used to determine whether the requirements have been met. To do this, the input domain should be partitioned into classes of values that the program will treat in a similar manner and for each class a representative element should be included in the test data. In addition, following should also be included in the data set: (1) boundary values (2) any non-extreme input values that would require special handling.

The output domain should be treated similarly.

Invalid input requires the same analysis as valid input.

- *The correctness, consistency and completeness of the requirements should also be analyzed* - Consider whether the correct problem is being solved, check for conflicts and inconsistencies among the requirements and consider the possibility of missing cases.

Design

The design document aids in programming, communication, and error analysis and test data generation. The requirements statement and the design document should together give the problem and the organization of the solution i.e. what the program will do and how it will be done.

The design document should contain:

- Principal data structures.
- Functions, algorithms, heuristics or special techniques used for processing.
- The program organization, how it will be modularized and categorized into external and internal interfaces.
- Any additional information.

Here the testing activities should consist of:

- *Analysis of design to check its completeness and consistency* - the total process should be analyzed to determine that no steps or special cases have been overlooked. Internal interfaces, I/O handling and data structures should specially be checked for inconsistencies.
- *Analysis of design to check whether it satisfies the requirements* - check whether both requirements and design document contain the same form, format, units used for input and output and also that all functions listed in the requirement document have been included in the design document. Selected test data which is generated during the requirements analysis phase should be manually simulated to determine whether the design will yield the expected values.
- *Generation of test data based on the design* - The tests generated should cover the structure as well as the internal functions of the design like the data structures, algorithm, functions, heuristics and general program structure etc. Standard extreme and special values should be included and expected output should be recorded in the test data.
- Reexamination and refinement of the test data set generated at the requirements analysis phase.

The first two steps should also be performed by some colleague and not only the designer/developer.

Programming/Construction

Here the main testing points are:

- *Check the code for consistency with design* - the areas to check include modular structure, module interfaces, data structures, functions, algorithms and I/O handling.
- *Perform the Testing process in an organized and systematic manner* with test runs dated, annotated and saved. A plan or schedule can be used as a checklist to help the programmer organize testing efforts. If errors are found and changes made to the program, all tests involving the erroneous segment (including those which resulted in success previously) must be rerun and recorded.
- *Asks some colleague for assistance* - Some independent party, other than the programmer of the specific part of the code, should analyze the development product at each phase. The programmer should explain the product to the party who will then question the logic and search for errors with a checklist to guide the search. This is needed to locate errors the programmer has overlooked.
- *Use available tools* - the programmer should be familiar with various compilers and interpreters available on the system for the implementation language being used because they differ in their error analysis and code generation capabilities.
- *Apply Stress to the Program* - Testing should exercise and stress the program structure, the data structures, the internal functions and the externally visible functions or functionality. Both valid and invalid data should be included in the test set.
- *Test one at a time* - Pieces of code, individual modules and small collections of modules should be exercised separately before they are integrated into the total program, one by one. Errors are easier to isolate when the no. of potential interactions should be kept small. Instrumentation-insertion of some code into the program solely to measure various program characteristics – can be useful here. A tester should perform array bound checks, check loop control variables,

determine whether key data values are within permissible ranges, trace program execution, and count the no. of times a group of statements is executed.

- *Measure testing coverage/When should testing stop?* - If errors are still found every time the program is executed, testing should continue. Because errors tend to cluster, modules appearing particularly error-prone require special scrutiny.

The metrics used to measure testing thoroughness include statement testing (whether each statement in the program has been executed at least once), branch testing (whether each exit from each branch has been executed at least once) and path testing (whether all logical paths, which may involve repeated execution of various segments, have been executed at least once). Statement testing is the coverage metric most frequently used as it is relatively simple to implement.

The amount of testing depends on the cost of an error. Critical programs or functions require more thorough testing than the less significant functions.

Operations and maintenance

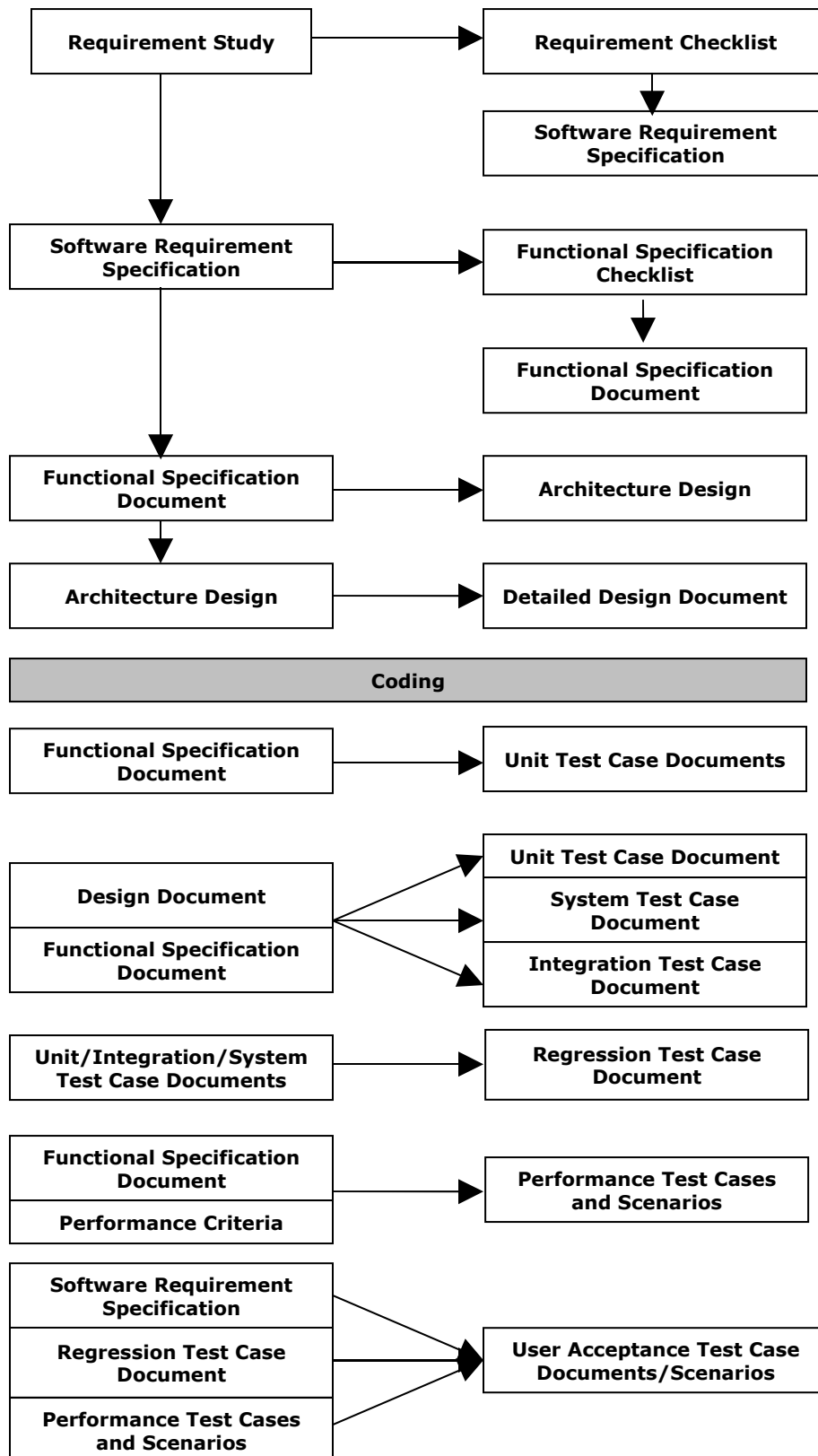
Corrections, modifications and extensions are bound to occur even for small programs and testing is required every time there is a change. Testing during maintenance is termed regression testing. The test set, the test plan, and the test results for the original program should exist. Modifications must be made to accommodate the program changes, and then all portions of the program affected by the modifications must be re-tested. After regression testing is complete, the program and test documentation must be updated to reflect the changes.

7. The Test Development Life Cycle (TDLC)

Usually, Testing is considered as a part of the System Development Life Cycle. With our practical experience, we framed this Test Development Life Cycle.

The diagram does not depict where and when you write your Test Plan and Strategy documents. But, it is understood that before you begin your testing activities these documents should be ready. Ideally, when the Project Plan and Project Strategy are being made, this is the time when the Test Plan and Test Strategy documents are also made.

Test Development Life Cycle (TDLC)



8. When should Testing stop?

"When to stop testing" is one of the most difficult questions to a test engineer.

The following are few of the common Test Stop criteria:

1. All the high priority bugs are fixed.
2. The rate at which bugs are found is too small.
3. The testing budget is exhausted.
4. The project duration is completed.
5. The risk in the project is under acceptable limit.

Practically, we feel that the decision of stopping testing is based on the level of the risk acceptable to the management. As testing is a never ending process we can never assume that 100 % testing has been done, we can only minimize the risk of shipping the product to client with X testing done. The risk can be measured by Risk analysis but for small duration / low budget / low resources project, risk can be deduced by simply: -

- Measuring Test Coverage.
- Number of test cycles.
- Number of high priority bugs.

9. Verification Strategies

What is 'Verification'?

Verification is the process of evaluating a system or component to determine whether the products of a given development phase satisfy the conditions imposed at the start of that phase.¹

What is the importance of the Verification Phase?

Verification process helps in detecting defects early, and preventing their leakage downstream. Thus, the higher cost of later detection and rework is eliminated.

9.1 Review

A process or meeting during which a work product, or set of work products, is presented to project personnel, managers, users, customers, or other interested parties for comment or approval.

¹

The main goal of reviews is to find defects. Reviews are a good compliment to testing to help assure quality. A few purposes' of SQA reviews can be as follows:

- Assure the quality of deliverable's before the project moves to the next stage.
- Once a deliverable has been reviewed, revised as required, and approved, it can be used as a basis for the next stage in the life cycle.

What are the various types of reviews?

Types of reviews include Management Reviews, Technical Reviews, Inspections, Walkthroughs and Audits.

Management Reviews

Management reviews are performed by those directly responsible for the system in order to monitor progress, determine status of plans and schedules, confirm requirements and their system allocation.

Therefore the main objectives of Management Reviews can be categorized as follows:

- Validate from a management perspective that the project is making progress according to the project plan.
- Ensure that deliverables are ready for management approvals.
- Resolve issues that require management's attention.
- Identify any project bottlenecks.
- Keeping project in Control.

Support decisions made during such reviews include Corrective actions, Changes in the allocation of resources or changes to the scope of the project

In management reviews the following Software products are reviewed:

Audit Reports

Contingency plans

Installation plans

Risk management plans

Software Q/A

The participants of the review play the roles of Decision-Maker, Review Leader, Recorder, Management Staff, and Technical Staff.

Technical Reviews

Technical reviews confirm that product Conforms to specifications, adheres to regulations, standards, guidelines, plans, changes are properly implemented, changes affect only those system areas identified by the change specification.

The main objectives of Technical Reviews can be categorized as follows:

- Ensure that the software confirms to the organization standards.
- Ensure that any changes in the development procedures (design, coding, testing) are implemented per the organization pre-defined standards.

In technical reviews, the following Software products are reviewed

- Software requirements specification
- Software design description
- Software test documentation
- Software user documentation
- Installation procedure
- Release notes

The participants of the review play the roles of Decision-maker, Review leader, Recorder, Technical staff.

What is Requirement Review?

A process or meeting during which the requirements for a system, hardware item, or software item are presented to project personnel, managers, users, customers, or other interested parties for comment or approval. Types include system requirements review, software requirements review.

Who is involved in Requirement Review?

- Product management leads Requirement Review. Members from every affected department participates in the review

Input Criteria

Software requirement specification is the essential document for the review. A checklist can be used for the review.

Exit Criteria

Exit criteria include the filled & completed checklist with the reviewers' comments & suggestions and the re-verification whether they are incorporated in the documents.

What is Design Review?

A process or meeting during which a system, hardware, or software design is presented to project personnel, managers, users, customers, or other interested parties for

comment or approval. Types include critical design review, preliminary design review, and system design review.

Who involve in Design Review?

- QA team member leads design review. Members from development team and QA team participate in the review.

Input Criteria

Design document is the essential document for the review. A checklist can be used for the review.

Exit Criteria

Exit criteria include the filled & completed checklist with the reviewers' comments & suggestions and the re-verification whether they are incorporated in the documents.

What is Code Review?

A meeting at which software code is presented to project personnel, managers, users, customers, or other interested parties for comment or approval.

Who is involved in Code Review?

- QA team member (In case the QA Team is only involved in Black Box Testing, then the Development team lead chairs the review team) leads code review. Members from development team and QA team participate in the review.

Input Criteria

The Coding Standards Document and the Source file are the essential documents for the review. A checklist can be used for the review.

Exit Criteria

Exit criteria include the filled & completed checklist with the reviewers' comments & suggestions and the re-verification whether they are incorporated in the documents.

9.2 Walkthrough

A static analysis technique in which a designer or programmer leads members of the development team and other interested parties through a segment of documentation or

code, and the participants ask questions and make comments about possible errors, violation of development standards, and other problems.

The objectives of Walkthrough can be summarized as follows:

- Detect errors early.
- Ensure (re)established standards are followed:
- Train and exchange technical information among project teams which participate in the walkthrough.
- Increase the quality of the project, thereby improving morale of the team members.

The participants in Walkthroughs assume one or more of the following roles:

- a) Walk-through leader
- b) Recorder
- c) Author
- d) Team member

To consider a review as a systematic walk-through, a team of at least two members shall be assembled. Roles may be shared among the team members. The walk-through leader or the author may serve as the recorder. The walk-through leader may be the author.

Individuals holding management positions over any member of the walk-through team shall not participate in the walk-through.

Input to the walk-through shall include the following:

- a) A statement of objectives for the walk-through
- b) The software product being examined
- c) Standards that are in effect for the acquisition, supply, development, operation, and/or maintenance of the software product

Input to the walk-through may also include the following:

- d) Any regulations, standards, guidelines, plans, and procedures against which the software product is to be inspected
- e) Anomaly categories

The walk-through shall be considered complete when

- a) The entire software product has been examined
- b) Recommendations and required actions have been recorded
- c) The walk-through output has been completed

9.3 Inspection

A static analysis technique that relies on visual examination of development products to detect errors, violations of development standards, and other problems. Types include code inspection; design inspection, Architectural inspections, Test ware inspections etc.

The participants in Inspections assume one or more of the following roles:

- a) Inspection leader
- b) Recorder
- c) Reader
- d) Author
- e) Inspector

All participants in the review are inspectors. The author shall not act as inspection leader and should not act as reader or recorder. Other roles may be shared among the team members. Individual participants may act in more than one role.

Individuals holding management positions over any member of the inspection team shall not participate in the inspection.

Input to the inspection shall include the following:

- a) A statement of objectives for the inspection
- b) The software product to be inspected
- c) Documented inspection procedure
- d) Inspection reporting forms
- e) Current anomalies or issues list

Input to the inspection may also include the following:

- f) Inspection checklists
- g) Any regulations, standards, guidelines, plans, and procedures against which the software product is to be inspected
- h) Hardware product specifications
- i) Hardware performance data
- j) Anomaly categories

The individuals may make additional reference material available responsible for the software product when requested by the inspection leader.

The purpose of the exit criteria is to bring an unambiguous closure to the inspection meeting. The exit decision shall determine if the software product meets the inspection exit criteria and shall prescribe any appropriate rework and verification. Specifically, the inspection team shall identify the software product disposition as one of the following:

<http://www.SofTReL.org>

- a) *Accept with no or minor rework.* The software product is accepted as is or with only minor rework. (For example, that would require no further verification).
- b) *Accept with rework verification.* The software product is to be accepted after the inspection leader or a designated member of the inspection team (other than the author) verifies rework.
- c) *Re-inspect.* Schedule a re-inspection to verify rework. At a minimum, a re-inspection shall examine the software product areas changed to resolve anomalies identified in the last inspection, as well as side effects of those changes.

10. Testing Types and Techniques

Testing types

Testing types refer to different approaches towards testing a computer program, system or product. The two types of testing are *black box testing* and *white box testing*, which would both be discussed in detail in this chapter. Another type, termed as *gray box testing* or *hybrid testing* is evolving presently and it combines the features of the two types.

Testing Techniques

Testing techniques refer to different methods of testing particular features a computer program, system or product. Each testing type has its own testing techniques while some techniques combine the feature of both types. Some techniques are

- Error and anomaly detection technique
- Interface checking
- Physical units checking
- Loop testing (Discussed in detail in this chapter)
- Basis Path testing/McCabe's cyclomatic number(Discussed in detail in this chapter)
- Control structure testing(Discussed in detail in this chapter)
- Error Guessing(Discussed in detail in this chapter)
- Boundary Value analysis (Discussed in detail in this chapter)
- Graph based testing(Discussed in detail in this chapter)
- Equivalence partitioning(Discussed in detail in this chapter)
- Instrumentation based testing
- Random testing
- Domain testing

- Halstead's software science
- And many more

Some of these and many others would be discussed in the later sections of this chapter.

Difference between Testing Types and Testing Techniques

Testing types deal with what aspect of the computer software would be tested, while testing techniques deal with how a specific part of the software would be tested.

That is, testing types mean whether we are testing the function or the structure of the software. In other words, we may test each function of the software to see if it is operational or we may test the internal components of the software to check if its internal workings are according to specification.

On the other hand, 'Testing technique' means what methods or ways would be applied or calculations would be done to test a particular feature of a software (Sometimes we test the interfaces, sometimes we test the segments, sometimes loops etc.)

How to Choose a Black Box or White Box Test?

White box testing is concerned only with testing the software product; it cannot guarantee that the complete specification has been implemented. Black box testing is concerned only with testing the specification; it cannot guarantee that all parts of the implementation have been tested. Thus black box testing is testing against the specification and will discover faults of omission, indicating that part of the specification has not been fulfilled. White box testing is testing against the implementation and will discover faults of commission, indicating that part of the implementation is faulty. In order to completely test a software product both black and white box testing are required.

White box testing is much more expensive (In terms of resources and time) than black box testing. It requires the source code to be produced before the tests can be planned and is much more laborious in the determination of suitable input data and the determination if the software is or is not correct. It is advised to start test planning with a black box testing approach as soon as the specification is available. White box tests are to be planned as soon as the Low Level Design (LLD) is complete. The Low Level Design will address all the algorithms and coding style. The paths should then be

checked against the black box test plan and any additional required test cases should be determined and applied.

The consequences of test failure at initiative/requirements stage are very expensive. A failure of a test case may result in a change, which requires all black box testing to be repeated and the re-determination of the white box paths. The cheaper option is to regard the process of testing as one of quality assurance rather than quality control. The intention is that sufficient quality is put into all previous design and production stages so that it can be expected that testing will project the presence of very few faults, rather than testing being relied upon to discover any faults in the software, as in case of quality control. A combination of black box and white box test considerations is still not a completely adequate test rationale.

10.1 White Box Testing

What is WBT?

White box testing involves looking at the *structure* of the code. When you know the internal structure of a product, tests can be conducted to ensure that the internal operations performed according to the specification. And all internal components have been adequately exercised. In other word WBT tends to involve the coverage of the specification in the code.

Code coverage is defined in six types as listed below.

- Segment coverage – Each segment of code b/w control structure is executed at least once.
- Branch Coverage or Node Testing – Each branch in the code is taken in each possible direction at least once.
- Compound Condition Coverage – When there are multiple conditions, you must test not only each direction but also each possible combinations of conditions, which is usually done by using a ‘Truth Table’
- Basis Path Testing – Each independent path through the code is taken in a pre-determined order. This point will further be discussed in other section.
- Data Flow Testing (DFT) – In this approach you track the specific variables through each possible calculation, thus defining the set of intermediate paths through the code i.e., those based on each piece of code chosen to be tracked.

Even though the paths are considered independent, dependencies across multiple paths are not really tested for by this approach. DFT tends to reflect dependencies but it is mainly through sequences of data manipulation. This approach tends to uncover bugs like variables used but not initialize, or declared but not used, and so on.

- Path Testing – Path testing is where all possible paths through the code are defined and covered. This testing is extremely laborious and time consuming.
- Loop Testing – In addition to the above measures, there are testing strategies based on loop testing. These strategies relate to testing single loops, concatenated loops, and nested loops. Loops are fairly simple to test unless dependencies exist among the loop or b/w a loop and the code it contains.

What do we do in WBT?

In WBT, we use the *control structure* of the procedural design to derive test cases. Using WBT methods a tester can derive the test cases that

- Guarantee that all independent paths within a module have been exercised at least once.
- Exercise all logical decisions on their true and false values.
- Execute all loops at their boundaries and within their operational bounds
- Exercise internal data structures to ensure their validity.

White box testing (WBT) is also called *Structural* or *Glass box* testing.

Why WBT?

We do WBT because Black box testing is unlikely to uncover numerous sorts of defects in the program. These defects can be of the following nature:

- *Logic errors* and *incorrect assumptions* are inversely proportional to the probability that a program path will be executed. Errors tend to creep into our work when we design and implement functions, conditions or controls that are out of the program
- The logical flow of the program is sometimes counterintuitive, meaning that our unconscious assumptions about flow of control and data may lead to *design errors* that are uncovered only when path testing starts.

- *Typographical errors* are random, some of which will be uncovered by syntax checking mechanisms but others will go undetected until testing begins.

Skills Required

Talking theoretically, all we need to do in WBT is to define all logical paths, develop test cases to exercise them and evaluate results i.e. generate test cases to exercise the program logic exhaustively.

For this we need to know the program well i.e. We should know the specification and the code to be tested; related documents should be available too us .We must be able to tell the expected status of the program versus the actual status found at any point during the testing process.

Limitations

Unfortunately in WBT, exhaustive testing of a code presents certain logistical problems. Even for small programs, the number of possible logical paths can be very large. For instance, a 100 line C Language program that contains two nested loops executing 1 to 20 times depending upon some initial input after some basic data declaration. Inside the interior loop four if-then-else constructs are required. Then there are approximately 10^{14} logical paths that are to be exercised to test the program exhaustively. Which means that a magic test processor developing a single test case, execute it and evaluate results in one millisecond would require 3170 years working continuously for this exhaustive testing which is certainly impractical. Exhaustive WBT is impossible for large software systems. But that doesn't mean WBT should be considered as impractical. Limited WBT in which a limited no. of important logical paths are selected and exercised and important data structures are probed for validity, is both practical and WBT. It is suggested that white and black box testing techniques can be coupled to provide an approach that that validates the software interface selectively ensuring the correction of internal working of the software.

Tools used for White Box testing:

Few Test automation tool vendors offer white box testing tools which:

- 1) Provide run-time error and memory leak detection;
- 2) Record the exact amount of time the application spends in any given block of code for the purpose of finding inefficient code bottlenecks; and
- 3) Pinpoint areas of the application that have and have not been executed.

10.1.1 Basis Path Testing

Basis path testing is a *white box testing* technique first proposed by Tom McCabe. The Basis path method enables to derive a logical complexity measure of a procedural design and use this measure as a guide for defining a basis set of execution paths. Test Cases derived to exercise the basis set are guaranteed to execute every statement in the program at least one time during testing.

10.1.2 Flow Graph Notation

The flow graph depicts logical control flow using a diagrammatic notation. Each structured construct has a corresponding flow graph symbol.

10.1.3 Cyclomatic Complexity

Cyclomatic complexity is a software metric that provides a quantitative measure of the logical complexity of a program. When used in the context of a basis path testing method, the value computed for Cyclomatic complexity defines the number for independent paths in the basis set of a program and provides us an upper bound for the number of tests that must be conducted to ensure that all statements have been executed at least once.

An *independent path* is any path through the program that introduces at least one new set of processing statements or a new condition.

Computing Cyclomatic Complexity

Cyclomatic complexity has a foundation in graph theory and provides us with extremely useful software metric. Complexity is computed in one of the three ways:

1. The number of regions of the flow graph corresponds to the Cyclomatic complexity.
2. Cyclomatic complexity, $V(G)$, for a flow graph, G is defined as

$$V(G) = E - N + 2$$

Where E , is the number of flow graph edges, N is the number of flow graph nodes.

3. Cyclomatic complexity, $V(G)$ for a flow graph, G is also defined as:

$$V(G) = P + 1$$

Where P is the number of predicate nodes contained in the flow graph G .

10.1.4 Graph Matrices

The procedure for deriving the flow graph and even determining a set of basis paths is amenable to mechanization. To develop a software tool that assists in basis path testing, a data structure, called a *graph matrix* can be quite useful.

A *Graph Matrix* is a square matrix whose size is equal to the number of nodes on the flow graph. Each row and column corresponds to an identified node, and matrix entries correspond to connections between nodes.

10.1.5 Control Structure Testing

Described below are some of the variations of Control Structure Testing.

Condition Testing

Condition testing is a test case design method that exercises the logical conditions contained in a program module.

Data Flow Testing

The data flow testing method selects test paths of a program according to the locations of definitions and uses of variables in the program.

10.1.6 Loop Testing

Loop Testing is a white box testing technique that focuses exclusively on the validity of loop constructs. Four classes of loops can be defined: Simple loops, Concatenated loops, nested loops, and unstructured loops.

Simple Loops

The following sets of tests can be applied to simple loops, where 'n' is the maximum number of allowable passes through the loop.

1. Skip the loop entirely.
2. Only one pass through the loop.
3. Two passes through the loop.
4. 'm' passes through the loop where $m < n$.
5. $n-1$, n , $n+1$ passes through the loop.

Nested Loops

If we extend the test approach from simple loops to nested loops, the number of possible tests would grow geometrically as the level of nesting increases.

1. Start at the innermost loop. Set all other loops to minimum values.
 2. Conduct simple loop tests for the innermost loop while holding the outer loops at their minimum iteration parameter values. Add other tests for out-of-range or exclude values.
3. Work outward, conducting tests for the next loop, but keep all other outer loops at minimum values and other nested loops to "typical" values.
4. Continue until all loops have been tested.

Concatenated Loops

Concatenated loops can be tested using the approach defined for simple loops, if each of the loops is independent of the other. However, if two loops are concatenated and the loop counter for loop 1 is used as the initial value for loop 2, then the loops are not independent.

Unstructured Loops

Whenever possible, this class of loops should be redesigned to reflect the use of the structured programming constructs.

10.2 Black Box Testing

Black box is a test design method. Black box testing treats the system as a "black-box", so it doesn't explicitly use Knowledge of the internal structure. Or in other words the Test engineer need not know the internal working of the "Black box".

It focuses on the functionality part of the module.

Some people like to call black box testing as behavioral, functional, opaque-box, and closed-box. While the term black box is most popularly use, many people prefer the terms "behavioral" and "structural" for black box and white box respectively. Behavioral test design is slightly different from black-box test design because the use of internal knowledge isn't strictly forbidden, but it's still discouraged.

Personally we feel that there is a trade off between the approaches used to test a product using white box and black box types.

There are some bugs that cannot be found using only black box or only white box. If the test cases are extensive and the test inputs are also from a large sample space then it is always possible to find majority of the bugs through black box testing.

Tools used for Black Box testing:

Many tool vendors have been producing tools for automated black box and automated white box testing for several years. The basic functional or regression testing tools capture the results of black box tests in a script format. Once captured, these scripts can be executed against future builds of an application to verify that new functionality hasn't disabled previous functionality.

Advantages of Black Box Testing

- Tester can be non-technical.

- This testing is most likely to find those bugs as the user would find.
- Testing helps to identify the vagueness and contradiction in functional specifications.
- Test cases can be designed as soon as the functional specifications are complete

Disadvantages of Black Box Testing

- Chances of having repetition of tests that are already done by programmer.
- The test inputs needs to be from large sample space.
- It is difficult to identify all possible inputs in limited testing time. So writing test cases is slow and difficult

Chances of having unidentified paths during this testing

10.2.1 Graph Based Testing Methods

Software testing begins by creating a graph of important objects and their relationships and then devising a series of tests that will cover the graph so that each objects and their relationships and then devising a series of tests that will cover the graph so that each object and relationship is exercised and error is uncovered.

10.2.2 Error Guessing

Error Guessing comes with experience with the technology and the project. Error Guessing is the art of guessing where errors can be hidden. There are no specific tools and techniques for this, but you can write test cases depending on the situation: Either when reading the functional documents or when you are testing and find an error that you have not documented.

10.2.3 Boundary Value Analysis

Boundary Value Analysis (BVA) is a test data selection technique (Functional Testing technique) where the extreme values are chosen. Boundary values include maximum, minimum, just inside/outside boundaries, typical values, and error values. The hope is that, if a system works correctly for these special values then it will work correctly for all values in between.

- Extends equivalence partitioning
 - Test both sides of each boundary
 - Look at output boundaries for test cases too
 - Test min, min-1, max, max+1, typical values
-
- BVA focuses on the boundary of the input space to identify test cases
 - Rational is that errors tend to occur near the extreme values of an input variable

There are two ways to generalize the BVA techniques:

1. By the number of variables
 - For n variables: BVA yields $4n + 1$ test cases.
2. By the kinds of ranges
 - Generalizing ranges depends on the nature or type of variables
 - NextDate has a variable Month and the range could be defined as {Jan, Feb, ...Dec}
 - $\text{Min} = \text{Jan}$, $\text{Min} + 1 = \text{Feb}$, etc.
 - Triangle had a declared range of {1, 20,000}
 - Boolean variables have extreme values True and False but there is no clear choice for the remaining three values

Advantages of Boundary Value Analysis

1. Robustness Testing - Boundary Value Analysis plus values that go beyond the limits
2. $\text{Min} - 1$, Min , $\text{Min} + 1$, Nom , $\text{Max} - 1$, Max , $\text{Max} + 1$
3. Forces attention to exception handling
4. For strongly typed languages robust testing results in run-time errors that abort normal execution

Limitations of Boundary Value Analysis

BVA works best when the program is a function of several independent variables that represent bounded physical quantities

1. Independent Variables
 - NextDate test cases derived from BVA would be inadequate: focusing on the boundary would not leave emphasis on February or leap years
 - Dependencies exist with NextDate's Day, Month and Year
 - Test cases derived without consideration of the function
2. Physical Quantities
 - An example of physical variables being tested, telephone numbers - what faults might be revealed by numbers of 000-0000, 000-0001, 555-5555, 999-9998, 999-9999?

10.2.4 Equivalence Partitioning

Equivalence partitioning is a black box testing method that divides the input domain of a program into classes of data from which test cases can be derived.

EP can be defined according to the following guidelines:

1. If an input condition specifies a range, one valid and one two invalid classes are defined.
2. If an input condition requires a specific value, one valid and two invalid equivalence classes are defined.
3. If an input condition specifies a member of a set, one valid and one invalid equivalence class is defined.
4. If an input condition is Boolean, one valid and one invalid class is defined.

10.2.5 Comparison Testing

There are situations where independent versions of software be developed for critical applications, even when only a single version will be used in the delivered computer based system. It is these independent versions which form the basis of a black box testing technique called Comparison testing or back-to-back testing.

10.2.6 Orthogonal Array Testing

The Orthogonal Array Testing Strategy (OATS) is a systematic, statistical way of testing pair-wise interactions by deriving a suitable small set of test cases (from a large number of possibilities).

11. Designing Test Cases

There are various techniques in which you can design test cases. For example, the below illustrated gives you an overview as to how you derive test cases using the basis path method:

The basis path testing method can be applied to a procedural design or to source code.

The following steps can be applied to derive the basis set:

1. Use the design or code as a foundation, draw corresponding flow graph.
2. Determine the Cyclomatic complexity of the resultant flow graph.
3. Determine a basis set of linearly independent paths.
4. Prepare test cases that will fore execution of each path in the basis set.

Let us now see how to design test cases in a generic manner:

1. Understand the requirements document.

2. Break the requirements into smaller requirements (if it improves your testability).
3. For each Requirement, decide what technique you should use to derive the test cases. For example, if you are testing a Login page, you need to write test cases basing on error guessing and also negative cases for handling failures.
4. Have a Traceability Matrix as follows:

Requirement No (In RD)	Requirement	Test Case No

What this Traceability Matrix provides you is the coverage of Testing. Keep filling in the Traceability matrix when you complete writing test case's for each requirement.

12. Validation Phase

The Validation Phase falls into picture after the software is ready or when the code is being written. There are various techniques and testing types that can be appropriately used while performing the testing activities. Let us examine a few of them.

12.1 Unit Testing

This is a typical scenario of Manual Unit Testing activity-

A Unit is allocated to a Programmer for programming. Programmer has to use 'Functional Specifications' document as input for his work.

Programmer prepares 'Program Specifications' for his Unit from the Functional Specifications. Program Specifications describe the programming approach, coding tips for the Unit's coding.

Using these 'Program specifications' as input, Programmer prepares 'Unit Test Cases' document for that particular Unit. A 'Unit Test Cases Checklist' may be used to check the completeness of Unit Test Cases document.

'Program Specifications' and 'Unit Test Cases' are reviewed and approved by Quality Assurance Analyst or by peer programmer.

The programmer implements some functionality for the system to be developed. The same is tested by referring the unit test cases. While testing that functionality if any defects have been found, they are recorded using the defect logging tool whichever is applicable. The programmer fixes the bugs found and tests the same for any errors.

Stubs and Drivers

A software application is made up of a number of 'Units', where output of one 'Unit' goes as an 'Input' of another Unit. e.g. A 'Sales Order Printing' program takes a 'Sales Order' as an input, which is actually an output of 'Sales Order Creation' program.

Due to such interfaces, independent testing of a Unit becomes impossible. But that is what we want to do; we want to test a Unit in isolation! So here we use 'Stub' and 'Driver'.

A 'Driver' is a piece of software that drives (invokes) the Unit being tested. A driver creates necessary 'Inputs' required for the Unit and then invokes the Unit.

A Unit may reference another Unit in its logic. A 'Stub' takes place of such subordinate unit during the Unit Testing. A 'Stub' is a piece of software that works similar to a unit which is referenced by the Unit being tested, but it is much simpler than the actual unit. A Stub works as a 'Stand-in' for the subordinate unit and provides the minimum required behavior for that unit.

Programmer needs to create such 'Drivers' and 'Stubs' for carrying out Unit Testing.

Both the Driver and the Stub are kept at a minimum level of complexity, so that they do not induce any errors while testing the Unit in question.

Example - For Unit Testing of 'Sales Order Printing' program, a 'Driver' program will have the code which will create Sales Order records using hardcoded data and then call 'Sales Order Printing' program. Suppose this printing program uses another unit which calculates Sales discounts by some complex calculations. Then call to this unit will be replaced by a 'Stub', which will simply return fix discount data.

Unit Test Cases

It must be clear by now that preparing Unit Test Cases document (referred to as UTC hereafter) is an important task in Unit Testing activity. Having an UTC, which is complete with every possible test case, leads to *complete* Unit Testing and thus gives an assurance of defect-free Unit at the end of Unit Testing stage. So let us discuss about how to prepare a UTC.

Think of following aspects while preparing Unit Test Cases –

- ❖ Expected Functionality: Write test cases against each functionality that is expected to be provided from the Unit being developed.

e.g. If an SQL script contains commands for creating one table and altering another table then test cases should be written for testing creation of one table and alteration of another.

It is important that User Requirements should be traceable to Functional Specifications, Functional Specifications be traceable to Program Specifications and Program Specifications be traceable to Unit Test Cases. Maintaining such traceability ensures that the application fulfills User Requirements.

- ❖ Input values:

- Every input value: Write test cases for each of the inputs accepted by the Unit.
e.g. If a Data Entry Form has 10 fields on it, write test cases for all 10 fields.
 - Validation of input: Every input has certain validation rule associated with it. Write test cases to validate this rule. Also, there can be cross-field validations in which one field is enabled depending upon input of another field. Test cases for these should not be missed.
e.g. A combo box or list box has a valid set of values associated with it.
A numeric field may accept only positive values.
An email address field must have ampersand (@) and period (.) in it.
A 'Sales tax code' entered by user must belong to the 'State' specified by the user.
 - Boundary conditions: Inputs often have minimum and maximum possible values. Do not forget to write test cases for them.
e.g. A field that accepts 'percentage' on a Data Entry Form should be able to accept inputs only from 1 to 100.
 - Limitations of data types: Variables that hold the data have their value limits depending upon their data types. In case of computed fields, it is very important to write cases to arrive at an upper limit value of the variables.
 - Computations: If any calculations are involved in the processing, write test cases to check the arithmetic expressions with all possible combinations of values.
- ❖ Output values: Write test cases to generate scenarios, which will produce all types of output values that are expected from the Unit.
e.g. A Report can display one set of data if user chooses a particular option and another set of data if user chooses a different option. Write test cases to check each of these outputs. When the output is a result of some calculations being performed or some formulae being used, then approximations play a major role and must be checked.
- ❖ Screen / Report Layout: Screen Layout or web page layout and Report layout must be tested against the requirements. It should not happen that the screen or the report looks beautiful and perfect, but user wanted something entirely different! It should ensure that pages and screens are consistent.
- ❖ Path coverage: A Unit may have conditional processing which results in various paths the control can traverse through. Test case must be written for each of these paths.

- ❖ Assumptions: A Unit may assume certain things for it to function. For example, a Unit may need a database to be open. Then test case must be written to check that the Unit reports error if such assumptions are not met.
- ❖ Transactions: In case of database applications, it is important to make sure that transactions are properly designed and no way inconsistent data gets saved in the database.
- ❖ Abnormal terminations: Behavior of the Unit in case of abnormal termination should be tested.
- ❖ Error messages: Error messages should be short, precise and self-explanatory. They should be properly phrased and should be free of grammatical mistakes.

UTC Document

Given below is a simple format for UTC document.

Test Case No.	Test Case purpose	Procedure	Expected Result	Actual result
ID which can be referred to in other documents like 'Traceability Matrix', Root Cause Analysis of Defects etc.	What to test	How to test	What should happen	What actually happened? This column can be omitted when Defect Recording Tool is used.

Note that as this is a sample, we have not provided columns for Pass/Fail and Remarks.

Example:

Let us say we want to write UTC for a Data Entry Form below:

Item No.		Item Master Form
Item Name		
Item Price		
Item No.		
Item No.		

Given below are some of the Unit Test Cases for the above Form:

Test Case No.	Test Case purpose	Procedure	Expected Result	Actual result
---------------	-------------------	-----------	-----------------	---------------

1	Item no. to start by 'A' or 'B'.	1.Create a new record. 2.Type Item no. starting with 'A'. 3.Type item no. starting with 'B'. 4.Type item no. starting with any character other than 'A' and 'B'.	2,3. Should get accepted and control should move to next field. 4. Should not get accepted. An error message should be displayed and control should remain in Item no. field.	
2.	Item Price to be between 1000 to 2000 if Item no. starts with 'A'.	1.Create a new record with Item no. starting with 'A'. 2.Specify price < 1000 3.Specify price >2000. 4.Specify price = 1000. 5.Specify price = 2000. 6.Specify price between 1000 and 2000.	2,3.Error should get displayed and control should remain in Price field. 4,5,6.Should get accepted and control should move to next field.	

UTC Checklist

UTC checklist may be used while reviewing the UTC prepared by the programmer. As any other checklist, it contains a list of questions, which can be answered as either a 'Yes' or a 'No'. The 'Aspects' list given in Section 4.3 above can be referred to while preparing UTC checklist.

e.g. Given below are some of the checkpoints in UTC checklist –

1. Are test cases present for all form field validations?
2. Are boundary conditions considered?
3. Are Error messages properly phrased?

Defect Recording

Defect Recording can be done on the same document of UTC, in the column of 'Expected Results'. This column can be duplicated for the next iterations of Unit Testing.

Defect Recording can also be done using some tools like Bugzilla, in which defects are stored in the database.

Defect Recording needs to be done with care. It should be able to indicate the problem in clear, unambiguous manner, and reproducing of the defects should be easily possible from the defect information.

Conclusion

Exhaustive Unit Testing filters out the defects at an early stage in the Development Life Cycle. It proves to be cost effective and improves Quality of the Software before the smaller pieces are put together to form an application as a whole. Unit Testing should be done sincerely and meticulously, the efforts are paid well in the long run.

12.2 Integration Testing

Integration testing is a systematic technique for constructing the program structure while at the same time conducting tests to uncover errors associated with interfacing. The objective is to take unit tested components and build a program structure that has been dictated by design.

Usually, the following methods of Integration testing are followed:

1. Top-down Integration approach.
2. Bottom-up Integration approach.

12.2.1 Top-Down Integration

Top-down integration testing is an incremental approach to construction of program structure. Modules are integrated by moving downward through the control hierarchy, beginning with the main control module. Modules subordinate to the main control module are incorporated into the structure in either a depth-first or breadth-first manner.

1. The Integration process is performed in a series of five steps:
2. The main control module is used as a test driver and stubs are substituted for all components directly subordinate to the main control module.
3. Depending on the integration approach selected subordinate stubs are replaced one at a time with actual components.
4. Tests are conducted as each component is integrated.
5. On completion of each set of tests, stub is replaced with the real component.
6. Regression testing may be conducted to ensure that new errors have not been introduced.

12.2.2 Bottom-Up Integration

Bottom-up integration testing begins construction and testing with atomic modules (i.e. components at the lowest levels in the program structure). Because components are integrated from the bottom up, processing required for components subordinate to a given level is always available and the need for stubs is eliminated.

1. A Bottom-up integration strategy may be implemented with the following steps:

<http://www.SofTReL.org>

2. Low level components are combined into clusters that perform a specific software sub function.
3. A driver is written to coordinate test case input and output.
4. The cluster is tested.

Drivers are removed and clusters are combined moving upward in the program structure.

12.3 System Testing

System testing concentrates on testing the complete system with a variety of techniques and methods. System Testing comes into picture after the Unit and Integration Tests.

12.3.1 Compatibility Testing

Compatibility Testing concentrates on testing whether the given application goes well with third party tools, software or hardware platform.

For example, you have developed a web application. The major compatibility issue is, the web site should work well in various browsers. Similarly when you develop applications on one platform, you need to check if the application works on other operating systems as well. This is the main goal of Compatibility Testing.

Before you begin compatibility tests, our sincere suggestion is that you should have a cross reference matrix between various software's, hardware based on the application requirements. For example, let us suppose you are testing a web application. A sample list can be as follows:

Hardware	Software	Operating System
Pentium – II, 128 MB RAM	IE 4.x, Opera, Netscape	Windows 95
Pentium – III, 256 MB RAM	IE 5.x, Netscape	Windows XP
Pentium – IV, 512 MB RAM	Mozilla	Linux

Compatibility tests are also performed for various client/server based applications where the hardware changes from client to client.

Compatibility Testing is very crucial to organizations developing their own products. The products have to be checked for compliance with the competitors of the third party tools, hardware, or software platform. E.g. A Call center product has been built for a solution with X product but there is a client interested in using it with Y product; then the issue of compatibility arises. It is of importance that the product is compatible with varying platforms. Within the same platform, the organization has to be watchful that with each new release the product has to be tested for compatibility.

A good way to keep up with this would be to have a few resources assigned along with their routine tasks to keep updated about such compatibility issues and plan for testing when and if the need arises.

By the above example it is not intended that companies which are not developing products do not have to cater for this type of testing. There case is equally existent, if an application uses standard software then would it be able to run successfully with the newer versions too? Or if a website is running on IE or Netscape, what will happen when it is opened through Opera or Mozilla. Here again it is best to keep these issues in mind and plan for compatibility testing in parallel to avoid any catastrophic failures and delays.

12.3.2 Recovery Testing

Recovery testing is a system test that focuses the software to fall in a variety of ways and verifies that recovery is properly performed. If it is automatic recovery then re-initialization, check pointing mechanisms, data recovery and restart should be evaluated for correctness. If recovery requires human intervention, the mean-time-to-repair (MTTR) is evaluated to determine whether it is within acceptable limits.

12.3.3 Usability Testing

Usability is the degree to which a user can easily learn and use a product to achieve a goal. Usability testing is the system testing which attempts to find any human-factor problems. A simpler description is testing the software from a users' point of view. Essentially it means testing software to prove/ensure that it is user-friendly, as distinct from testing the functionality of the software. In practical terms it includes ergonomic considerations, screen design, standardization etc.

The idea behind usability testing is to have actual users perform the tasks for which the product was designed. If they can't do the tasks or if they have difficulty performing the tasks, the UI is not adequate and should be redesigned. It should be remembered that usability testing is just one of the many techniques that serve as a basis for evaluating the UI in a user-centered approach. Other techniques for evaluating a UI include inspection methods such as heuristic evaluations, expert reviews, card-sorting, matching test or Icon intuitiveness evaluation, cognitive walkthroughs. Confusion regarding usage of the term can be avoided if we use 'usability evaluation' for the generic term and reserve 'usability testing' for the specific evaluation method based on user performance. Heuristic Evaluation and Usability Inspection or cognitive walkthrough does not involve real users.

It often involves building prototypes of parts of the user interface, having representative users perform representative tasks and seeing if the appropriate users can perform the tasks. In other techniques such as the inspection methods, it is not performance, but someone's opinion of how users might perform that is offered as evidence that the UI is acceptable or not. This distinction between performance and opinion about performance is crucial. Opinions are subjective. Whether a sample of users can accomplish what they want or not is objective. Under many circumstances it is more useful to find out if users can do what they want to do rather than asking someone.

PERFORMING THE TEST

1. Get a person who fits the user profile. Make sure that you are not getting someone who has worked on it.
2. Sit them down in front of a computer, give them the application, and tell them a small scenario, like: "Thank you for volunteering making it easier for users to find what they are looking for. We would like you to answer several questions. There is no right or wrong answers. What we want to learn is why you make the choices you do, what is confusing, why choose one thing and not another, etc. Just talk us through your search and let us know what you are thinking. We have a recorder which is going to capture what you say, so you will have to tell us what you are clicking on as you also tell us what you are thinking. Also think aloud when you are stuck somewhere"
3. Now don't speak anything. Sounds easy, but see if you actually can shut up.
4. Watch them use the application. If they ask you something, tell them you're not there. Then shut up again.
5. Start noting all the things you will have to change.
6. Afterwards ask them what they thought and note them down.
7. Once the whole thing is done thank the volunteer.

TOOLS AVAILABLE FOR USABILITY TESTING

- [ErgoLight Usability Software](#) offers comprehensive GUI quality solutions for the professional Windows application developer. ErgoLight offers solutions for developers of Windows applications for testing and evaluating their usability.
- [WebMetrics Tool Suite](#) from **National Institute of Standards and Technology** contains rapid, remote, and automated tools to help in producing usable web sites. The Web Static Analyzer Tool (WebSAT) checks the html of a web page against numerous usability guidelines. The output from WebSAT

consists of identification of potential usability problems, which should be investigated further through user testing. The Web Category Analysis Tool (WebCAT) lets the usability engineer quickly construct and conduct a simple category analysis across the web.

- **Bobby** from **Center for Applied Special Technology** is a web-based public service offered by CAST that analyzes web pages for their accessibility to people with disabilities as well as their compatibility with various browsers.
- **DRUM** from **Serco Usability Services** is a tool, which has been developed by close cooperation between Human Factors professionals and software engineers to provide a broad range of support for video-assisted observational studies.
- [Form Testing Suite](#) from **Corporate Research and Advanced Development, Digital Equipment Corporation** Provides a test suite developed to test various web browsers. The test results section provides a description of the tests.

USABILITY LABS

- The **Usability Center** (ULAB) is a full service organization, which provides a "Street-Wise" approach to usability risk management and product usability excellence. It has custom designed ULAB facilities.
- **Usability Sciences Corporation** has a usability lab in Dallas consisting of two large offices separated by a one way mirror. The test room in each lab is equipped with multiple video cameras, audio equipment, as well as everything a user needs to operate the program. The video control and observation room features five monitors, a video recorder with special effects switching, two-way audio system, remote camera controls, a PC for test log purposes, and a telephone for use as a help desk.
- **UserWorks, Inc.** (formerly Man-Made Systems) is a consulting firm in the Washington, DC area specializing in the design of user-product interfaces. UserWorks does analyses, market research, user interface design, rapid prototyping, product usability evaluations, competitive testing and analyses, ergonomic analyses, and human factors contract research. UserWorks offers several portable usability labs (audio-video data collection systems) for sale or rent and an observational data logging software product for sale.
- **Lodestone Research** has usability-testing laboratory with state of the art audio and visual recording and testing equipment. All equipment has been designed to be portable so that it can be taken on the road. The lab consists of a test room and an observation/control room that can seat as many as ten observers. A-V

equipment includes two (soon to be 3) fully controllable SVHS cameras, capture/feed capabilities for test participant's PC via scan converter and direct split signal (to VGA "slave" monitors in observation room), up to eight video monitors and four VCA monitors for observer viewing, mixing/editing equipment, and "wiretap" capabilities to monitor and record both sides of telephone conversation (e.g., if participant calls customer support).

- **Online Computer Library Center, Inc** provides insight into the usability test laboratory. It gives an overview of the infrastructure as well as the process being used in the laboratory.

END GOALS OF USABILITY TESTING

To summarize the goals, it can be said that it makes the software more user friendly. The end result will be:

- Better quality software.
- Software is easier to use.
- Software is more readily accepted by users.
- Shortens the learning curve for new users.

12.3.4 Security Testing

Security testing attempts to verify that protection mechanisms built into a system will, in fact, protect it from improper penetration. During Security testing, password cracking, unauthorized entry into the software, network security are all taken into consideration.

12.3.5 Stress Testing

Stress testing executes a system in a manner that demands resources in abnormal quantity, frequency, or volume. The following types of tests may be conducted during stress testing;

- Special tests may be designed that generate ten interrupts per second, when one or two is the average rate.
- Input data rates may be increased by an order of magnitude to determine how input functions will respond.
- Test Cases that require maximum memory or other resources.
- Test Cases that may cause excessive hunting for disk-resident data.
- Test Cases that may cause thrashing in a virtual operating system.

12.3.6 Performance Testing

Performance testing of a Web site is basically the process of understanding how the Web application and its operating environment respond at various user load levels. In general, we want to measure the **Response Time**, **Throughput**, and **Utilization** of the Web site while simulating attempts by virtual users to simultaneously access the site. One of the main objectives of performance testing is to maintain a Web site with low response time, high throughput, and low utilization.

Response Time

Response Time is the delay experienced when a request is made to the server and the server's response to the client is received. It is usually measured in units of time, such as seconds or milliseconds. Generally speaking, Response Time increases as the inverse of unutilized capacity. It increases slowly at low levels of user load, but increases rapidly as capacity is utilized. Figure 1 demonstrates such typical characteristics of Response Time versus user load.

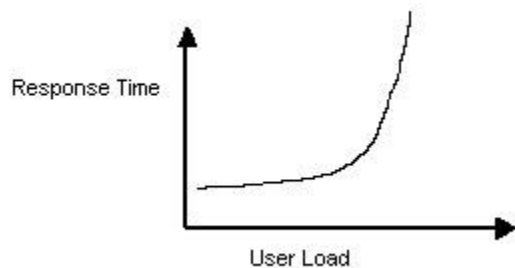


Figure1. Typical characteristics of latency versus user load

The sudden increase in response time is often caused by the maximum utilization of one or more system resources. For example, most Web servers can be configured to start up a fixed number of threads to handle concurrent user requests. If the number of concurrent requests is greater than the number of threads available, any incoming requests will be placed in a queue and will wait for their turn to be processed. Any time spent in a queue naturally adds extra wait time to the overall Response Time.

To better understand what Response Time means in a typical Web farm, we can divide response time into many segments and categorize these segments into two major types: network response time and application response time. Network response time refers to the time it takes for data to travel from one server to another. Application response time

is the time required for data to be processed within a server. Figure 2 shows the different response time in the entire process of a typical Web request.

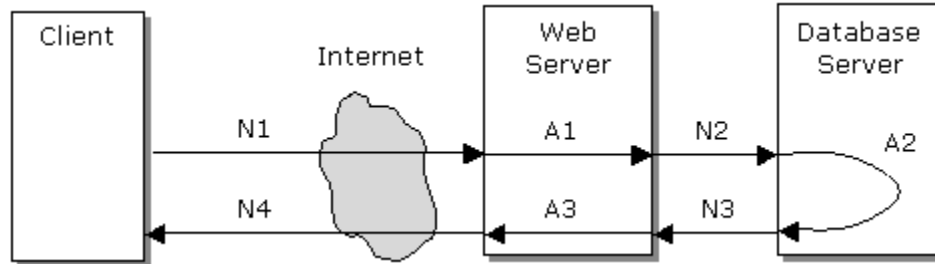


Figure 2 shows the different response time in the entire process of a typical Web request.

$$\text{Total Response Time} = (N1 + N2 + N3 + N4) + (A1 + A2 + A3)$$

Where N_x represents the network Response Time and A_x represents the application Response Time.

In general, the Response Time is mainly constrained by $N1$ and $N4$. This Response Time represents the method your clients are using to access the Internet. In the most common scenario, e-commerce clients access the Internet using relatively slow dial-up connections. Once Internet access is achieved, a client's request will spend an indeterminate amount of time in the Internet cloud shown in Figure 2 as requests and responses are funneled from router to router across the Internet.

To reduce these networks Response Time ($N1$ and $N4$), one common solution is to move the servers and/or Web contents closer to the clients. This can be achieved by hosting your farm of servers or replicating your Web contents with major Internet hosting providers who have redundant high-speed connections to major public and private Internet exchange points, thus reducing the number of network routing hops between the clients and the servers.

Network Response Times $N2$ and $N3$ usually depend on the performance of the switching equipment in the server farm. When traffic to the back-end database grows, consider upgrading the switches and network adapters to boost performance.

Reducing application Response Times ($A1$, $A2$, and $A3$) is an art form unto itself because the complexity of server applications can make analyzing performance data and performance tuning quite challenging. Typically, multiple software components interact on the server to service a given request. Response time can be introduced by any of the components. That said, there are ways you can approach the problem:

- First, your application design should minimize round trips wherever possible. Multiple round trips (client to server or application to database) multiply transmission and resource acquisition Response time. Use a single round trip wherever possible.
- You can optimize many server components to improve performance for your configuration. Database tuning is one of the most important areas on which to focus. Optimize stored procedures and indexes.
- Look for contention among threads or components competing for common resources. There are several methods you can use to identify contention bottlenecks. Depending on the specific problem, eliminating a resource contention bottleneck may involve restructuring your code, applying service packs, or upgrading components on your server. Not all resource contention problems can be completely eliminated, but you should strive to reduce them wherever possible. They can become bottlenecks for the entire system.
- Finally, to increase capacity, you may want to upgrade the server hardware (scaling up), if system resources such as CPU or memory are stretched out and have become the bottleneck. Using multiple servers as a cluster (scaling out) may help to lessen the load on an individual server, thus improving system performance and reducing application latencies.

Throughput

Throughput refers to the number of client requests processed within a certain unit of time. Typically, the unit of measurement is requests per second or pages per second. From a marketing perspective, throughput may also be measured in terms of visitors per day or page views per day, although smaller time units are more useful for performance testing because applications typically see peak loads of several times the average load in a day.

As one of the most useful metrics, the throughput of a Web site is often measured and analyzed at different stages of the design, develop, and deploy cycle. For example, in the process of capacity planning, throughput is one of the key parameters for determining the hardware and system requirements of a Web site. Throughput also plays an important role in identifying performance bottlenecks and improving application and system performance. Whether a Web farm uses a single server or multiple servers, throughput statistics show similar characteristics in reactions to various user load levels. Figure 3 demonstrates such typical characteristics of throughput versus user load.

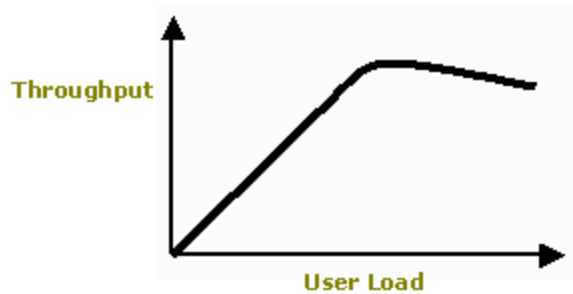


Figure 3. Typical characteristics of throughput versus user load

As Figure 3 illustrates, the throughput of a typical Web site increases proportionally at the initial stages of increasing load. However, due to limited system resources, throughput cannot be increased indefinitely. It will eventually reach a peak, and the overall performance of the site will start degrading with increased load. Maximum throughput, illustrated by the peak of the graph in Figure 3, is the maximum number of user requests that can be supported concurrently by the site in the given unit of time.

Note that it is sometimes confusing to compare the throughput metrics for your Web site to the published metrics of other sites. The value of maximum throughput varies from site to site. It mainly depends on the complexity of the application. For example, a Web site consisting largely of static HTML pages may be able to serve many more requests per second than a site serving dynamic pages. As with any statistic, throughput metrics can be manipulated by selectively ignoring some of the data. For example, in your measurements, you may have included separate data for all the supporting files on a page, such as graphic files. Another site's published measurements might consider the overall page as one unit. As a result, throughput values are most useful for comparisons within the same site, using a common measuring methodology and set of metrics.

In many ways, throughput and Response time are related, as different approaches to thinking about the same problem. In general, sites with high latency will have low throughput. If you want to improve your throughput, you should analyze the same criteria as you would to reduce latency. Also, measurement of throughput without consideration of latency is misleading because latency often rises under load before throughput peaks. This means that peak throughput may occur at a latency that is unacceptable from an application usability standpoint. This suggests that Performance reports include a cut-off value for Response time, such as: 250 requests/second @ 5 seconds maximum Response time

Utilization

Utilization refers to the usage level of different system resources, such as the server's CPU(s), memory, network bandwidth, and so forth. It is usually measured as a percentage of the maximum available level of the specific resource. Utilization versus user load for a Web server typically produces a curve, as shown in Figure 4.

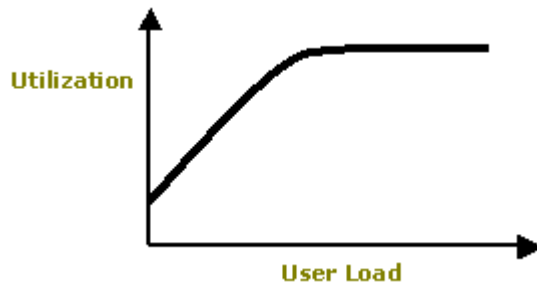


Figure 4. Typical characteristics of utilization versus user load

As Figure 4 illustrates, utilization usually increases proportionally to increasing user load. However, it will top off and remain at a constant when the load continues to build up.

If the specific system resource tops off at 100-percent utilization, it's very likely that this resource has become the performance bottleneck of the site. Upgrading the resource with higher capacity would allow greater throughput and lower latency—thus better performance. If the measured resource does not top off close to 100-percent utilization, it is probably because one or more of the other system resources have already reached their maximum usage levels. They have become the performance bottleneck of the site.

To locate the bottleneck, you may need to go through a long and painstaking process of running performance tests against each of the suspected resources, and then verifying if performance is improved by increasing the capacity of the resource. In many cases, performance of the site will start deteriorating to an unacceptable level well before the major system resources, such as CPU and memory, are maximized. For example, Figure 5 illustrates a case where response time rises sharply to 45 seconds when CPU utilization has reached only 60 percent.

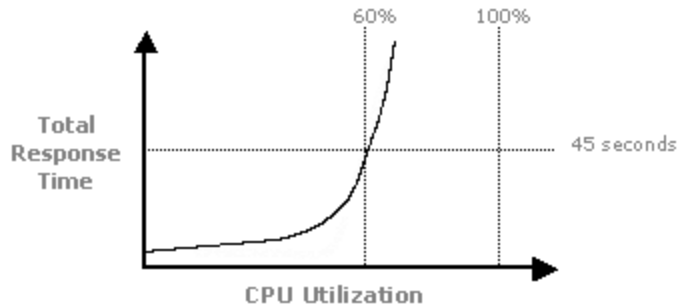


Figure 5. An example of Response Time versus utilization

As Figure 5 demonstrates, monitoring the CPU or memory utilization alone may not always indicate the true capacity level of the server farm with acceptable performance.

Applications

While most traditional applications are designed to respond to a single user at any time, most Web applications are expected to support a wide range of concurrent users, from a dozen to a couple thousand or more. As a result, performance testing has become a critical component in the process of deploying a Web application. It has proven to be most useful in (but not limited to) the following areas:

- Capacity planning
- Bug fixing

Capacity Planning

How do you know if your server configuration is sufficient to support two million visitors per day with average response time of less than five seconds? If your company is projecting a business growth of 200 percent over the next two months, how do you know if you need to upgrade your server or add more servers to the Web farm? Can your server and application support a six-fold traffic increase during the Christmas shopping season?

Capacity planning is about being prepared. You need to set the hardware and software requirements of your application so that you'll have sufficient capacity to meet anticipated and unanticipated user load.

One approach in capacity planning is to load-test your application in a testing (staging) server farm. By simulating different load levels on the farm using a Web application performance testing tool such as WAS, you can collect and analyze the test results to better understand the performance characteristics of the application. Performance charts such as those shown in Figures 1, 3, and 4 can then be generated to show the expected Response Time, throughput, and utilization at these load levels.

In addition, you may also want to test the scalability of your application with different hardware configurations. For example, load testing your application on servers with one, two, and four CPUs respectively would help to determine how well the application scales with symmetric multiprocessor (SMP) servers. Likewise, you should load test your application with different numbers of clustered servers to confirm that your application scales well in a cluster environment.

Although performance testing is as important as functional testing, it's often overlooked. Since the requirements to ensure the performance of the system is not as straightforward as the functionalities of the system, achieving it correctly is more difficult.

The effort of performance testing is addressed in two ways:

- Load testing
- Stress testing

Load testing

Load testing is a much used industry term for the effort of performance testing. Here load means the number of users or the traffic for the system. Load testing is defined as the testing to determine whether the system is capable of handling anticipated number of users or not.

In Load Testing, the virtual users are simulated to exhibit the real user behavior as much as possible. Even the user think time such as how users will take time to think before inputting data will also be emulated. It is carried out to justify whether the system is performing well for the specified limit of load.

For example, Let us say an online-shopping application is anticipating 1000 concurrent user hits at peak period. In addition, the peak period is expected to stay for 12 hrs. Then the system is load tested with 1000 virtual users for 12 hrs. These kinds of tests are carried out in levels: first 1 user, 50 users, and 100 users, 250 users, 500 users and so on till the anticipated limit are reached. The testing effort is closed exactly for 1000 concurrent users.

The objective of load testing is to check whether the system can perform well for specified load. The system may be capable of accommodating more than 1000 concurrent users. But, validating that is not under the scope of load testing. No attempt is made to determine how many more concurrent users the system is capable of servicing. Table 1 illustrates the example specified.

Stress testing

Stress testing is another industry term of performance testing. Though load testing & Stress testing are used synonymously for performance-related efforts, their goal is different.

Unlike load testing where testing is conducted for specified number of users, stress testing is conducted for the number of concurrent users beyond the specified limit. The objective is to identify the maximum number of users the system can handle before breaking down or degrading drastically. Since the aim is to put more stress on system, think time of the user is ignored and the system is exposed to excess load. The goals of load and stress testing are listed in Table 2. Refer to table 3 for the inference drawn through the Performance Testing Efforts.

Let us take the same example of online shopping application to illustrate the objective of stress testing. It determines the maximum number of concurrent users an online system can service which can be beyond 1000 users (specified limit). However, there is a possibility that the maximum load that can be handled by the system may found to be same as the anticipated limit. The Table<##>illustrates the example specified.

Stress testing also determines the behavior of the system as user base increases. It checks whether the system is going to degrade gracefully or crash at a shot when the load goes beyond the specified limit.

Table 1: Load and stress testing of illustrative example

Types of Testing	Number of Concurrent users	Duration
Load Testing	1 User → 50 Users → 100 Users → 250 Users → 500 Users..... → 1000Users	12 Hours
Stress Testing	1 User → 50 Users → 100 Users → 250 Users → 500 Users..... → 1000Users → Beyond 1000 Users..... → Maximum Users	12 Hours

Table 2: Goals of load and stress testing

Types of testing	Goals
Load testing	• Testing for anticipated user base • Validates whether system is capable of handling load under specified limit
Stress testing	• Testing beyond the anticipated user base • Identifies the maximum load a system can handle • Checks whether the system degrades gracefully or crashes at a shot

Table 3: Inference drawn by load and stress testing

Type of Testing	Inference
Load Testing	Whether system Available? If yes, is the available system is stable?
Stress Testing	Whether system is Available? If yes, is the available system is stable? If Yes, is it moving towards Unstable state? When the system is going to break down or degrade drastically?

Conducting performance testing manually is almost impossible. Load and stress tests are carried out with the help of automated tools. Some of the popular tools to automate performance testing are listed in table 4.

Table 4: Load and stress testing tools

Tools	Vendor
LoadRunner	Mercury Interactive Inc
Astra load test	Mercury Interactive Inc
Silk performer	Segue
WebLoad	Radview Software
QALoad	CompuWare
e-Load	Empirix Software
eValid	Software research Inc
WebSpray	CAI network
TestManager	Rational

Web application center test	Microsoft technologies
OpenLoad	OpenDemand
ANTS	Red Gate Software
OpenSTA	Open source
Astra Loadtest	Mercury interactive Inc
WAPT	Novasoft Inc
Sitestress	Webmaster solutions
Quatiumpro	Quatium technologies
Easy WebLoad	PrimeMail Inc

Bug Fixing

Some errors may not occur until the application is under high user load. For Example, memory leaks can exacerbate server or application problems sustaining high load. Performance testing helps to detect and fix such problems before launching the application. It is therefore recommended that developers take an active role in performance testing their applications, especially at different major milestones of the development cycle.

12.3.7 Content Management Testing

‘Content Management’ has gained a predominant importance after the Web applications took a major part of our lives. What is Content Management? As the name denotes, it is managing the content. How do they work? Let us take a common example. You are in China and you wanted to open the Yahoo! Chinese version. When you choose *Chinese* version on the main page of Yahoo! You get to see the entire content in Chinese. Yahoo! Would strategically plan and have various servers for various languages. When you choose a particular version of the page, the request is redirected to the server which manages the Chinese content page. The Content Management systems help is placing content for various purposes and also help in displaying when the request comes in.

Content Management Testing involves:

1. Testing the distribution of the content.
2. Request, Response Time's.
3. Content display on various browsers and operating systems.
4. Load distribution on the servers.

In fact all the performance related testing should be performed for each version of the web application which uses the content management servers.

12.3.8 Regression Testing

Regression testing as the name suggests is used to test / check the effect of changes made in the code.

<http://www.SofTReL.org>

Most of the time the testing team is asked to check last minute changes in the code just before making a release to the client, in this situation the testing team needs to check only the affected areas.

So in short for the regression testing the testing team should get the input from the development team about the nature / amount of change in the fix so that testing team can first check the fix and then the side effects of the fix.

In my present organization we too faced the same problem. So we made a regression bucket (this is a simple excel sheet containing the test cases that we need think assure us of bare minimum functionality) this bucket is run every time before the release.

In fact the regression testing is the testing in which maximum automation can be done. The reason being the same set of test cases will be run on different builds multiple times.

But again the extent of automation depends on whether the test cases will remain applicable over the time, In case the automated test cases do not remain applicable for some amount of time then test engineers will end up in wasting time to automate and don't get enough out of automation.

- What is Regression testing?

Regression Testing is retesting unchanged segments of application. It involves rerunning tests that have been previously executed to ensure that the same results can be achieved currently as were achieved when the segment was last tested.

The selective retesting of a [software](#) system that has been modified to ensure that any [bugs](#) have been fixed and that no other previously working functions have failed as a result of the reparations and that newly added features have not created problems with previous versions of the software. Also referred to as verification testing, regression testing is initiated after a [programmer](#) has attempted to fix a recognized problem or has added [source code](#) to a program that may have inadvertently introduced errors. It is a quality control measure to ensure that the newly modified code still complies with its specified requirements and that unmodified code has not been affected by the maintenance activity.

- What do you do during Regression testing?
 - Rerunning of previously conducted tests

- Reviewing previously prepared manual procedures
- Comparing the current test results with the previously executed test results

- What are the tools available for Regression testing?

Although the process is simple i.e. the test cases that have been prepared can be used and the expected results are also known, if the process is not automated it can be very time-consuming and tedious operation.

Some of the tools available for regression testing are:

Record and Playback tools – Here the previously executed scripts can be rerun to verify whether the same set of results are obtained. E.g. Rational Robot

- What are the end goals of Regression testing?

- To ensure that the unchanged system segments function properly
- To ensure that the previously prepared manual procedures remain correct after the changes have been made to the application system
- To verify that the data dictionary of data elements that have been changed is correct

Regression testing as the name suggests is used to test / check the effect of changes made in the code.

Most of the time the testing team is asked to check the last minute changes in the code just before making a release to the client, in this situation the testing team needs to check only the affected areas.

So in short for the regression testing the testing team should get the input from the development team about the nature / amount of change in the fix so that testing team can first check the fix and then the affected areas.

In my present organization we too faced the same problem. So we made a regression bucket (this is a simple excel sheet containing the test cases that we need think assure us of bare minimum functionality) this bucket is run every time before the release.

In fact the regression testing is the testing in which maximum automation can be done. The reason being the same set of test cases will be run on different builds multiple times.

But again the extent of automation depends on whether the test cases will remain applicable over the time, In case the automated test cases do not remain applicable for some amount of time then test engineers will end up in wasting time to automate and don't get enough out of automation.

12.4 Alpha Testing

A software prototype stage when the software is first available for run. Here the software has the core functionalities in it but complete functionality is not aimed at. It would be able to accept inputs and give outputs. Usually the most used functionalities (parts of code) are developed more. The test is conducted at the developer's site only.

In a software development cycle, depending on the functionalities the number of alpha phases required is laid down in the project plan itself.

During this, the testing is not a through one, since only the prototype of the software is available. Basic installation – uninstallation tests, the completed core functionalities are tested. The functionality complete area of the Alpha stage is got from the project plan document.

Aim

- is to identify any serious errors
- to judge if the indented functionalities are implemented
- to provide to the customer the feel of the software

A through understanding of the product is done now. During this phase, the test plan and test cases for the beta phase (the next stage) is created. The errors reported are documented internally for the testers and developers reference. No issues are usually reported and recorded in any of the defect management/bug trackers

Role of test lead

- Understand the system requirements completely.
- Initiate the preparation of test plan for the beta phase.

Role of the tester

- to provide input while there is still time to make significant changes as the design evolves.
- Report errors to developers

12.5 User Acceptance Testing

User Acceptance testing occurs just before the software is released to the customer. The end-users along with the developers perform the User Acceptance Testing with a certain set of test cases and typical scenarios.

12.6 Installation Testing

Installation testing is often the most under tested area in testing. This type of testing is performed to ensure that all Installed features and options function properly. It is also performed to verify that all necessary components of the application are, indeed, installed.

Installation testing should take care of the following points: -

1. To check if while installing product checks for the dependent software / patches say Service pack3.
2. The product should check for the version of the same product on the target machine, say the previous version should not be over installed on the newer version.
3. Installer should give a default installation path say "C:\programs\."
4. Installer should allow user to install at location other then the default installation path.
5. Check if the product can be installed "Over the Network"
6. Installation should start automatically when the CD is inserted.
7. Installer should give the remove / Repair options.
8. When uninstalling, check that all the registry keys, files, Dll, shortcuts, active X components are removed from the system.
9. Try to install the software without administrative privileges (login as guest).
10. Try installing on different operating system.

Try installing on system having non-compliant configuration such as less memory / RAM / HDD.

12.7 Beta Testing

The Beta testing is conducted at one or more customer sites by the end-user of the software. The beta test is a live application of the software in an environment that cannot be controlled by the developer.

The Software reaches beta stage when most of the functionalities are operating.

The software is tested in customer's environment, giving user the opportunity to exercise the software, find the errors so that they could be fixed before product release.

Beta testing is a detailed testing and needs to cover all the functionalities of the product and also the dependent functionality testing. It also involves the UI testing and documentation testing. Hence it is essential that this is planned well and the task accomplished. The test plan document has to be prepared before the testing phase is started, which clearly lays down the objectives, scope of test, tasks to be performed and the test matrix which depicts the schedule of testing.

Beta Testing Objectives

- Evaluate software technical content
- Evaluate software ease of use
- Evaluate user documentation draft
- Identify errors
- Report errors/findings

Role of a Test Lead

- Provide **Test Instruction Sheet** that describes items such as testing objectives, steps to follow, data to enter, functions to invoke.
- Provide feedback forms and comments.

Role of a tester

- Understand the software requirements and the testing objectives.
- Carry out the test cases

Report defects

13. Understanding Exploratory Testing

"Exploratory testing involves simultaneously learning, planning, running tests, and reporting / troubleshooting"

<http://www.SofTReL.org>

Results."**- Dr. Cem Kaner.**

"Exploratory testing is an interactive process of concurrent product exploration, test design and test execution. To the extent that the next test we do is influenced by the result of the last test we did, we are doing exploratory testing."

- James Bach.

Exploratory testing is defined as simultaneous test design, test execution and bug reporting. In this approach the tester explores the system (finding out what it is and then testing it) without having any prior test cases or test scripts. Because of this reason it also called as ad hoc testing, guerrilla testing or intuitive testing. But there is some difference between them. In operational terms, exploratory testing is an interactive process of concurrent product exploration, test design, and test execution. The outcome of an exploratory testing session is a set of notes about the product, failures found, and a concise record of how the product was tested. When practiced by trained testers, it yields consistently valuable and auditable results. Every tester performs this type of testing at one point or the other. This testing totally depends on the skill and creativity of the tester. Different testers can explore the system in different ways depending on their skills. Thus the tester has a very vital role to play in exploratory testing.

This approach of testing has also been advised by SWEBOK for testing since it might uncover the bugs, which the normal testing might not discover. A systematic approach of exploratory testing can also be used where there is a plan to attack the system under test. This systematic approach of exploring the system is termed Formalized exploratory testing.

Exploratory testing is a powerful approach in the field of testing. Yet this approach has not got the recognition and is often misunderstood and not gained the respect it needs. In many situations it can be more productive than the scripted testing. But the real fact is that all testers do practice this methodology sometime or the other, most often unknowingly!

Exploratory testing believes in concurrent phases of product exploration, test design and test execution. It is categorized under Black-box testing. It is basically a free-style testing approach where you do not begin with the usual procedures of elaborate test plans and test steps. The test plan and strategy is very well in the tester's mind. The tester asks the right question to the product / application and judges the

outcome. During this phase he is actually learning the product as he tests it. It is interactive and creative. A conscious plan by the tester gives good results.

Human beings are unique and think differently, with a new set of ideas emerging. A tester has the basic skills to listen, read, think and report. Exploratory testing is just trying to exploit this and structure it down. The richness of this process is only limited to the breadth and depth of our imagination and the insight into the product under test.

How does it differ from the normal test procedures?

The definition of exploratory testing conveys the difference. In the normal testing style, the test process is planned well in advance before the actual testing begins. Here the test design is separated from the test execution phase. Many a times the test design and test execution is entrusted on different persons.

Exploratory testing should not be confused with the dictionary meaning of “ad-hoc”. Ad hoc testing normally refers to a process of improvised, impromptu bug searching. By definition, anyone can do ad hoc testing. The term “exploratory testing”--by Dr. Cem Kaner, in Testing Computer Software--refers to a sophisticated, systematic, thoughtful approach to ad hoc testing.

What is formalized Exploratory Testing?

A structured and reasoned approach to exploratory testing is termed as Formalized Exploratory Testing. This approach consists of specific tasks, objectives, and deliverables that make it a systematic process.

Using the systematic approach (i.e. the formalize approach) an outline of what to attack first, its scope, the time required to be spent etc is achieved. The approach might be using simple notes to more descriptive charters to some vague scripts. By using the systematic approach the testing can be more organized focusing at the goal to be reached. Thus solving the problem where the pure Exploratory Testing might drift away from the goal.

When we apply Exploratory Planning to Testing, we create Exploratory planning.

The formalized approach used for the Exploratory Testing can vary depending on the various criteria like the resource, time, the knowledge of the application

available etc. Depending on these criteria, the approach used to attack the system will also vary. It may involve creating the outlines on the notepad to more sophisticated way by using charters etc. Some of the formal approaches used for Exploratory Testing can be summarized as follows.

✓ Identify the application domain.

The exploratory testing can be performed by identifying the application domain. If the tester has good knowledge of domain, then it would be easier to test the system without having any test cases. If the tester were well aware of the domain, it would help analyzing the system faster and better. His knowledge would help in identifying the various workflows that usually exist in that domain. He would also be able to decide what are the different scenarios and which are most critical for that system. Hence he can focus his testing depending on the scenarios required. If a QA lead is trying to assign the tester to a task, it is advisable that the tester identifies the person who has the domain knowledge of that system for ET.

For example, consider software has been built to generate the invoices for its customers depending on the number of the units of power that has been consumed. In such a case exploratory testing can be done by identifying the domain of the application. A tester who has experience of the billing systems for the energy domain would fit better than one who does not have any knowledge. The tester who has knowledge in the application domain knows the terminology used as well the scenarios that would be critical to the system. He would know the ways in which various computations are done. In such a case, tester with good knowledge would be familiar to the terms like to line item, billing rate, billing cycle and the ways in which the computation of invoice would be done. He would explore the system to the best and takes lesser time. If the tester does not have domain knowledge required, then it would take time to understand the various workflows as well the terminology used. He might not be able to focus on critical areas rather focus on the other areas.

✓ Identify the purpose.

Another approach to Exploratory Testing is by identifying the purpose of the system i.e. What is that system used for. By identifying the purpose try to analyze to what extent it is used. The effort can be more focused by identifying the purpose.

For example, consider software developed to be used in Medical operations. In such case care should be taken that the software build is 100% defect free. Hence the effort that needs to be focused is more and care should be taken that the various workflows involved are covered.

On the other hand, if the software build is to provide some entertainment then the criticality is lesser. Thus effort that needs to be focused varies. Identifying the purpose of the system or the application to be tested helps to a great extent.

✓ Identify the primary and secondary functions.

Primary Function: Any function so important that, in the estimation of a normal user, its inoperability or impairment would render the product unfit for its purpose. A function is primary if you can associate it with the purpose of the product *and* it is essential to that purpose. Primary functions define the product. For example, the function of adding text to a document in Microsoft Word is certainly so important that the product would be useless without it. Groups of functions, taken together, may constitute a primary function, too. For example, while perhaps no single function on the drawing toolbar of Word would be considered primary, the entire toolbar might be primary. If so, then most of the functions on that toolbar should be operable in order for the product to pass Certification.

Secondary Function or contributing function: Any function that contributes to the utility of the product, but is not a primary function. Even though contributing functions are not primary, their inoperability *may* be grounds for refusing to grant Certification. For example, users may be technically able to do useful things with a product, even if it has an “Undo” function that never works, but most users will find that intolerable. Such a failure would violate fundamental expectations about how Windows products should work.

Thus by identifying the primary function and secondary functions for the system, testing can be done where more focus and effort can be given to Primary functions compared to the secondary functions.

Example: Consider a web based application developed for online shopping. For such an application we can identify the primary functions and secondary functions and go ahead with Exploratory Testing. The main functionality of that application is that the items selected by the user need to be properly added to the shopping cart and price to be paid is properly calculated. If there is online payment, then security is also an aspect. These can be considered as the primary functions.

Whereas the bulletin board provided or the mail functionality provided are considered as the secondary functions. Thus testing to be performed is more focused at the primary functions rather than on the secondary functions. If the primary functions do not work as required then the main intention of having the application is lost.

✓ Identify the workflows.

Identifying the workflows for testing any system without any scripted test cases can be considered as one of the best approaches used. The workflows are nothing but a visual representation of the scenarios as the system would behave for any given input. The workflows can be simple flow charts or Data Flow Diagram's (DFD) or the something like state diagrams, use cases, models etc. The workflows will also help to identify the scope for that scenario. The workflows would help the tester to keep track of the scenarios for testing. It is suggested that the tester navigates through the application before he starts exploring. It helps the tester in identifying the various possible workflows and issues any found which he is comfortable can be discussed with the concerned team.

Example: Consider a web application used for online shopping. The application has various links on the web page. If tester is trying to test if the items that he is adding to cart are properly being added, then he should know the flow for the same. He should first identify the workflow for such a scenario. He needs to login and then select a category and identify the items and then add the item he would require. Thus without

knowing the workflow for such a scenario would not help the tester and in the process loses his time.

In case he is not aware of the system, try to navigate through the application once and get comfortable. Once the application is dully understood, it is easier to test and explore more bugs.

✓ Identify the break points.

Break points are the situations where the system starts behaving abnormally. It does not give the output it is supposed to give. So by identifying such situations also testing can be done. Use boundary values or invariance for finding the break points of the application. In most of the cases it is observed that system would work for normal inputs or outputs. Try to give input that might be the ideal situation or the worse situation.

Example: consider an application build to generate the reports for the accounts department of a company depending on the criteria given. In such cases try to select a worse case of report generation for all the employees for their service. The system might not behave normally in the situation.

Try to input a large input file to the application that provides the user to upload and save the data given.

Try to input 500 characters in the text box of the web application.

Thus by trying to identify the extreme conditions or the breakpoints would help the tester to uncover the hidden bugs. Such cases might not be covered in the normal scripted testing. Hence this helps in finding the bugs which might not covered in the normal testing.

✓ Check UI against Windows interface etc standards.

The Exploratory Testing can be performed by identifying the User interface standards. There are set standards laid down for the user interfaces that need to be developed. These user standards are nothing but the look and feel aspects of the interfaces the user interacts with. The user should be comfortable with any of the screens that (s)he working. These aspects help the end user to accept the system faster.

Example: For Web application,

- Is the background as per the standards? If the bright background is used, the user might not feel comfortable.
- What is size of the font used?
- Are the buttons of the required size and are they placed in the comfortable location.
- Sometimes the applications are developed to avoid usage of the scroll bar. The content can be seen with out the need to scroll.

By identifying the User standards, define an approach to test because the application developed should be user friendly for the user's usage. He should feel comfortable while using the system. The more familiar and easier the application for usage, faster the user feels comfortable to the system.

✓ Identify expected results.

The tester should know what he is testing for and expected output for the given input. Until and unless the aim of his testing is not known, there is no use of the testing done. Because the tester may not succeed in distinguishing the real error and normal workflow. First he needs to analyze what is the expected output for the scenario he is testing.

Example: Consider software used to provide the user with an interface to search for the employee name in the organization given some of the inputs like the first name or last name or his id etc. For such a scenario, the tester should identify the expected output for any combination of input values. If the input provided does not result in any data and shows a message "Error not data found". The tester should not misinterpret this as an error, because this might be as per requirement when no data is found. Instead for a given input, the message shown is "404- File not found", the tester should identify it as an error not a requirement. Thus he should be able to distinguish between an error and normal workflow.

✓ Identify the interfaces with other interfaces/external applications.

In the age of component development and maximum reusability, developers try to pick up the already developed components and integrate them. Thus, achieving the desired result in short time. In some

cases it would help the tester explore the areas where the components are coupled. The output of one component should be correctly sent to other component. Hence such scenarios or workflows need to be identified and explored more. More focus on some of the shown areas that are more error prone.

Example: consider the online shopping application. The user adds the items to his cart and proceeds to the payments details page. Here the items added, their quantity etc should be properly sent to the next module. If there is any error in any of the data transfer process, the pay details will not be correct and the user will be billed wrong. There by leading to a major error. In such a scenario, more focus is required in the interfaces.

There may be external interfaces, like the application is integrated with another application for the data. In such cases, focus should be more on the interface between the two applications. How data is being passed, is correct data being passed, if there is large data, is transfer of entire data done or is system behaving abnormally when there is large data are few points which should be addressed.

✓ Record failures

In exploratory testing, we do the testing without having any documented test cases. If a bug has been found, it is very difficult for us to test it after fix. This is because there are no documented steps to navigate to that particular scenario. Hence we need to keep track of the flow required to reach where a bug has been found. So while testing, it is important that at least the bugs that have been discovered are documented. Hence by recording failures we are able to keep track of work that has been done. This would also help even if the tester who was actually doing ET is not available. Since the document can be referred and list all the bugs that have been reported as well the flows for the same can be identified.

Example: for example consider the online shopping site. A bug has been found while trying to add the items of given category into the cart. If the tester can just document the flow as well as the error that has occurred,

it would help the tester himself or any other tester. It can be referred while testing the application after a fix.

✓ Document issues and question.

The tester trying to test an application using Exploratory Testing methodology should feel comfortable to test. Hence it is advisable that the tester navigates through the application once and notes any ambiguities or queries he might feel. He can even get the clarification on the workflows he is not comfortable. Hence by documenting all the issues and questions that have been found while scanning or navigating the application can help the tester have testing done without any loss in time.

✓ Decompose the main task into smaller tasks .The smaller ones to still smaller activities.

It is always easier to work with the smaller tasks when compared to large tasks. This is very useful in performing Exploratory Testing because lack of test cases might lead us to different routes. By having a smaller task, the scope as well as the boundary are confined which will help the tester to focus on his testing and plan accordingly.

If a big task is taken up for testing, as we explore the system, we might get deviated from our main goal or task. It might be hard to define boundaries if the application is a new one. With smaller tasks, the goal is known and hence the focus and the effort required can be properly planned.

Example: An application that provides email facility. The new users can register and use the application for the email. In such a scenario, the main task itself can be divided into smaller tasks. One task to check if the UI standards are met and it is user friendly. The other task is to test if the new users are able to register with the application and use email facility.

Thus the two tasks are smaller which will the corresponding groups to focus their testing process.

✓ Charter- states the goal and the tactics to be used.

Charter Summary:

- “Architecting the Charters” i.e. Test Planning
- Brief information / guidelines on:
 - Mission: Why do we test this?
 - What should be tested?
 - How to test (approach)?
 - What problems to look for?
- Might include guidelines on:
 - Tools to use
 - Specific Test Techniques or tactics to use
 - What risks are involved
 - Documents to examine
 - Desired output from the testing.

A charter can be simple one to more descriptive giving the strategies and outlines for the testing process.

Example: Test the application for report generation.

Or.

Test the application if the report is being generated for the date before 01/01/2000. Use the use cases models for identifying the workflows.

✓ Session Based Test Management(SBTM):

Session Based Test Management is a formalized approach that uses the concept of charters and the sessions for performing the ET.

A *session* is not a test case or bug report. It is the reviewable product produced by chartered and uninterrupted test effort. A session can last from 60 to 90 minutes, but there is no hard and fast rule on the time spent for testing. If a session lasts closer to 45 minutes, we call it a *short*

session. If it lasts closer to two hours, we call it a *long* session. Each session designed depends on the tester and the charter. After the session is completed, each session is debriefed. The primary objective in the debriefing is to understand and accept the session report. Another objective is to provide feedback and coaching to the tester. The debriefings would help the manager to plan the sessions in future and also to estimate the time required for testing the similar functionality.

The debriefing session is based on agenda called PROOF.

Past: What happened during the session?

Results: What was achieved during the session?

Outlook: What still needs to be done?

Obstacles: What got in the way of good testing?

Feeling: How does the tester feel about all this?

The time spent “on charter” and “on opportunity” is also noted. Opportunity testing is any testing that doesn’t fit the charter of the session. The tester is not restricted to his charter, and hence allowed to deviate from the goal specified if there is any scope of finding an error.

A session can be broadly classified into three tasks (namely the TBS metrics).

Session test up: Time required in setting up the application under test.

Test design and execution: Time required scanning the product and test.

Bug investigation and reporting: Time required finding the bugs and reporting to the concerned.

The entire session report consists of these sections:

- ✓ Session charter (includes a mission statement, and areas to be tested)
- ✓ Tester name(s)

- ✓ Date and time started
- ✓ Task breakdown (the TBS metrics)
- ✓ Data files
- ✓ Test notes
- ✓ Issues
- ✓ Bugs

For each session, a session sheet is made. The session sheet consist of the mission of testing, the tester details, duration of testing, the TBS metrics along with the data related to testing like the bugs, notes, issues etc. Data files if any used in the testing would also be enclosed. The data collected during different testing sessions are collected and exported to Excel or some database. All the sessions, the bugs reported etc can be tracked using the unique id associated with each. It is easy for the client as well to keep track. Thus this concept of testers testing in sessions and producing the required output which are trackable is called as Session based test management.

✓ Defect Driven Exploratory Testing:

Defect driven exploratory testing is another formalized approach used for ET.

Defect Driven Exploratory Testing (DDET) is a goal-oriented approach focused on the critical areas identified on the Defect analysis study based on Procedural Testing results.

In Procedural testing, the tester executes readily available test cases, which are written based on the requirement specifications. Although the test cases are executed completely, defects were found in the software while doing exploratory testing by just wandering through the product blindly. Just exploring the product without sight was akin to groping in the dark and did not help the testers unearth all the hidden bugs in the software as they were not very sure about the areas that needed to be explored in the software. A reliable basis was needed for exploring the software. Thus Defect driven exploratory testing is an idea of exploring that part of the product based on the results obtained during

procedural testing. After analyzing the defects found during the DDET process, it was found that these were the most critical bugs, which were camouflaged in the software and which if present could have made the software 'Not fit for Use'.

There are some pre requisites for DDET:

- In-depth knowledge of the product.
- Procedural Testing has to be carried out.
- Defect Analysis based on Scripted Tests.

Advantages of DDET:

- Tester has clear clues on the areas to be explored.
- Goal oriented approach , hence better results
- No wastage of time.

Where does Exploratory Testing Fit:

In general, ET is called for in any situation where it's not obvious what the next test should be, or when you want to go beyond the obvious tests. More specifically, freestyle Exploratory Testing fits in any of the following situations:

- ✓ You need to provide rapid feedback on a new product or feature.
- ✓ You need to learn the product quickly.
- ✓ You have already tested using scripts, and seek to diversify the testing.
- ✓ You want to find the single most important bug in the shortest time.
- ✓ You want to check the work of another tester by doing a brief independent Investigation.
- ✓ You want to investigate and isolate a particular defect.
- ✓ You want to investigate the status of a particular risk, in order to evaluate the need for scripted tests in that area.

Pros and Cons:

Pros

- ✓ Does not require extensive documentation.
- ✓ Responsive to changing scenarios.
- ✓ Under tight schedules, testing can be more focused depending on the bug rate or risks.
- ✓ Improved coverage.

Cons

- ✓ Dependent on the tester's skills.

- ✓ Test tracking not concrete.
- ✓ More prone to human error.

- ✓ No contingency plan if the tester is unavailable.

What specifics affect Exploratory Testing?

Here is a list that affects exploratory testing:

- The mission of the particular test session
- The tester skills, talents, preferences
- Available time and other resources
- The status of other testing cycles for the product
- How much the tester knows about the product

Mission

The goal of testing needs to be understood first before the work begins. This could be the overall mission of the test project or could be a particular functionality / scenario. The mission is achieved by asking the right questions about the product, designing tests to answer these questions and executing tests to get the answers. Often the tests do not completely answer, in such cases we need to explore. The test procedure is recorded (which could later form part of the scripted testing) and the result status too.

Tester

The tester needs to have a general plan in mind, though may not be very constrained. The tester needs to have the ability to design good test strategy, execute good tests, find important problems and report them. He simply has to think out of the box.

Time

Time available for testing is a critical factor. Time falls short due to the following reasons:

- Many a time in project life cycles, the time and resources required in creating the test strategy, test plan and design, execution and reporting is overlooked. Exploratory testing becomes useful since the test plan, design and execution happen together.
- Also when testing is essential on a short period of notice
- A new feature is implemented

- Change request come in much later stage of the cycle when much of the testing is done with

In such situations exploratory testing comes handy.

Practicing Exploratory Testing

A basic strategy of exploratory testing is to have a general plan of attack, but also allow yourself to deviate from it for short period of time.

In a session of exploratory testing, a set of test ideas, written notes (simple English or scripts) and bug reports are the results. This can be reviewed by the test lead / test manager.

➤ Test Strategy

It is important to identify the scope of the test to be carried. This is dependent on the project approach to testing. The test manager / test lead can decide the scope and convey the same to the test team.

➤ Test design and execution

The tester crafts the test by systematically exploring the product. He defines his approach, analyze the product, and evaluate the risk

➤ Documentation

The written notes / scripts of the tester are reviewed by the test lead / manager. These later form into new test cases or updated test materials.

Where does Exploratory Testing Fit?

Exploratory testing fits almost in any kind of testing projects, projects with rigorous test plans and procedures or in projects where testing is not dictated completely in advance. The situations where exploratory testing could fit in are:

- Need to provide a rapid feedback on a new feature implementation / product
- Little product knowledge and need to learn it quickly
- Product analysis and test planning
- Done with scripted testing and need to diversify more
- Improve the quality of existing test scripts
- Write new scripts

The basic rule is this: exploratory testing is called for any time the next test you should perform is not obvious, or when you want to go beyond the obvious.

A Good Exploratory Tester

Exploratory testing approach relies a lot on the tester himself. The tester actively controls the design of tests as they are performed and uses the information gained to design new and better ideas.

A good exploratory tester should

- Have the ability to design good tests, execute them and find important problems
- Should document his ideas and use them in later cycles.
- Must be able to explain his work
- Be a careful observer: Exploratory testers are more careful observers than novices and experienced scripted testers. Scripted testers need only observe what the script tells. Exploratory tester must watch for anything unusual or mysterious.
- Be a critical thinker: They are able to review and explain their logic, looking out for errors in their own thinking.
- Have diverse ideas so as to make new test cases and improve existing ones.

A good exploratory tester always asks himself, *what's the best test I can perform now?* They remain alert for new opportunities.

Advantages

Exploratory testing is advantageous when

- Rapid testing is essential
- Test case development time not available
- Need to cover high risk areas with more inputs
- Need to test software with little knowledge about the specifications
- Develop new test cases or improve the existing
- Drive out monotony of normal step – by - step test execution

Drawbacks

- Skilled tester required
- Difficult to quantize

Balancing Exploratory Testing with Scripted Testing

Exploratory testing relies on the tester and the approach he proceeds with. Pure scripted testing doesn't undergo much change with time and hence the power fades

away. In test scenarios where in repeatability of tests are required, automated scripts having an edge over exploratory approach. Hence it is important to achieve a balance between the two approaches and combine the two to get the best of both.

14. Understanding Scenario Based Testing

Scenario Based Tests (SBT) are best suited when you need to tests need to concentrate on the functionality of the application, than anything else.

Let us take an example, where you are testing an application which is quite old (a legacy application) and it is a banking application. This application has been built based on the requirements of the organization for various banking purposes. Now, this application will have continuous upgrades in the working (technology wise and business wise). What do you do to test the application?

Let us assume that the application is undergoing only functional changes and not the UI changes. The test cases should be updated for every release. Over a period of time, maintaining the test ware becomes a major set back. The Scenario Based Tests would help you here.

As per the requirements, the base functionality is stable and there are no UI changes. There are only changes with respect to the business functionality. As per the requirements and the situation, we clearly understand that only regression tests need to be run continuously as part of the testing phase. Over a period of time, the individual test cases would become difficult to manage. This is the situation where we use Scenarios for testing.

What do you do for deriving Scenarios?

We can use the following as the basis for deriving scenarios:

5. From the requirements, list out all the functionalities of the application.
6. Using a graph notation, draw depictions of various transactions which pass through various functionalities of the application.
7. Convert these depictions into scenarios.
8. Run the scenarios when performing the testing.

Will you use Scenario Based Tests only for Legacy application testing?

No. Scenario Based Tests are not only for legacy application testing, but for any application which requires you to concentrate more on the functional requirements. If you can plan out a perfect test strategy, then the Scenario Based Tests can be used for any application testing and for any requirements.

Scenario Based tests will be a good choice with a combination of various test types and techniques when you are testing projects which adopt UML (Unified Modeling Language) based development strategies.

You can derive scenarios based on the Use Case's. Use Case's provide good coverage of the requirements and functionality.

15. Understanding Agile Testing

The concept of Agile testing rests on the values of the Agile Alliance Values, which states that:

“We have come to value:
Individuals and interactions *over* processes and tools
Working software *over* comprehensive documentation
Customer collaboration *over* contract negotiation
Responding to change *over* following a plan

That is, while there is value in the items on the right, we value the items on the left more." - <http://www.agilemanifesto.org/>

What is Agile testing?

- 1) Agile testers treat the developers as their customer and follow the agile manifesto. The *Context driven testing principles* (explained in later part) act as a set of principles for the agile tester.
- 2) Or it can be treated as the testing methodology followed by testing team when an entire project follows agile methodologies. If so what is the role of a tester in such a fast paced methodology?)

Traditional QA seems to be totally at loggerheads with the Agile manifesto in the following regard where:

- Process and tools are a key part of QA and testing.
- QA people seem to love documentation.
- QA people want to see the written specification.
- And where is testing without a PLAN?

So the question arises is there a role for QA in Agile projects?

There answer is maybe but the roles and tasks are different.

<http://www.SofTReL.org>

In the first definition of Agile testing we described it as one following the Context driven principles.

The context driven principles which are guidelines for the agile tester are:

1. The value of any practice depends on its context.
2. There are good practices in context, but there are no best practices.
3. People, working together, are the most important part of any project's context.
4. Projects unfold over time in ways that are often not predictable.
5. The product is a solution. If the problem isn't solved, the product doesn't work.
6. Good software testing is a challenging intellectual process.
7. Only through judgment and skill, exercised cooperatively throughout the entire project, are we able to do the right things at the right times to effectively test our products.

<http://www.context-driven-testing.com/>

In the second definition we described Agile testing as a testing methodology adopted when an entire project follows Agile (development) Methodology. We shall have a look at the Agile development methodologies being practiced currently:

Agile Development Methodologies

- Extreme Programming (XP)
- Crystal
- Adaptive Software Development (ASD)
- Scrum
- Feature Driven Development (FDD)
- Dynamic Systems Development Method (DSDM)
- Xbreed

In a fast paced environment such as in Agile development the question then arises as to what is the “**Role**” of testing?

Testing is as relevant in an Agile scenario if not more than a traditional software development scenario.

<http://www.SofTReL.org>

Testing is the *Headlight* of the agile project showing where the project is standing now and the direction it is headed.

Testing provides the required and relevant information to the teams to take informed and precise decisions.

The testers in agile frameworks get involved in much more than finding “software bugs”, anything that can “*bug*” the potential user is a issue for them but testers don’t make the final call, it’s the entire team that discusses over it and takes a decision over a potential issues.

A firm belief of Agile practitioners is that any testing approach does not assure quality it’s the team that does (or doesn’t) do it, so there is a heavy emphasis on the skill and attitude of the people involved.

Agile Testing is not a game of “gotcha”, it’s about finding ways to set goals rather than focus on mistakes.

Among these Agile methodologies mentioned we shall look at XP (Extreme Programming) in detail, as this is the most commonly used and popular one.

The basic components of the **XP** practices are:

- Test- First Programming
- Pair Programming
- Short Iterations & Releases
- Refactoring
- User Stories
- Acceptance Testing

We shall discuss these factors in detail.

Test-First Programming

- Developers write unit tests before coding. It has been noted that this kind of approach motivates the coding, speeds coding and also and improves design results in better designs (with less coupling and more cohesion)
- It supports a practice called Refactoring (discussed later on).

- Agile practitioners prefer Tests (code) to Text (written documents) for describing system behavior. Tests are more precise than human language and they are also a lot more likely to be updated when the design changes. How many times have you seen design documents that no longer accurately described the current workings of the software? Out-of-date design documents look pretty much like up-to-date documents. Out-of-date tests fail.
- Many open source tools like xUnit have been developed to support this methodology.

Refactoring

- Refactoring is the practice changing a software system in such a way that it does not alter the external behavior of the code yet improves its internal structure.
- Traditional development tries to understand how all the code will work together in advance. This is the design. With agile methods, this difficult process of imagining what code might look like before it is written is avoided. Instead, the code is restructured as needed to maintain a coherent design. Frequent refactoring allows less up-front planning of design.
- Agile methods replace high-level design with frequent redesign (refactoring). Successful refactoring But it also requires a way of ensuring checking whether that the behavior wasn't inadvertently changed. That's where the tests come in.
- Make the simplest design that will work and add complexity only when needed and refactor as necessary.
- Refactoring requires unit tests to ensure that design changes (refactorings) don't break existing code.

Acceptance Testing

- Make up user experiences or User stories, which are short descriptions of the features to be coded.
- Acceptance tests verify the completion of user stories.
- Ideally they are written before coding.

With all these features and process included we can define a practice for Agile testing encompassing the following features.

- Conversational Test Creation
- Coaching Tests
- Providing Test Interfaces
- Exploratory Learning

Looking deep into each of these practices we can describe each of them as:

Conversational Test Creation

- Test case writing should be a collaborative activity including majority of the entire team. As the customers will be busy we should have someone representing the customer.
- Defining tests is a key activity that should include programmers and customer representatives.
- Don't do it alone.

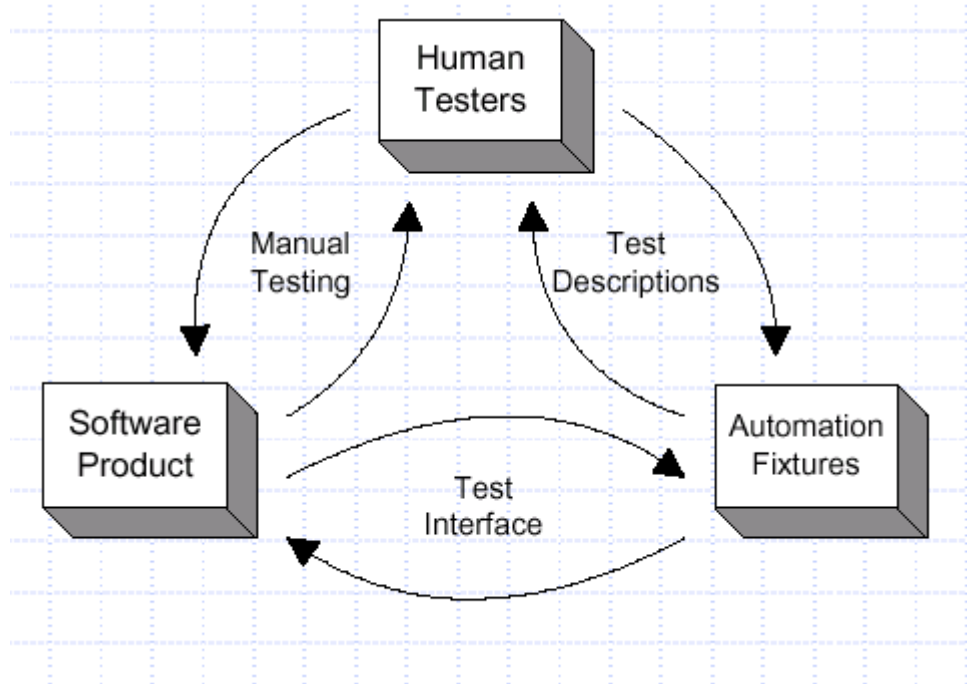
Coaching Tests

- A way of thinking about Acceptance Tests.
- Turn user stories into tests.
- Tests should provide Goals and guidance, Instant feedback and Progress measurement
- Tests should be in specified in a format that is clear enough that users/ customers can understand and that is specific enough that it can be executed
- Specification should be done by example.

Providing Test Interfaces

- Developers are responsible for providing the fixtures that automate coaching tests
- In most cases XP teams are adding test interfaces to their products, rather than using external test tools

Test Interaction Model



Exploratory Learning

- Plan to explore, learn and understand the product with each iteration.
- Look for bugs, missing features and opportunities for improvement.
- We don't understand software until we have used it.

We believe that Agile Testing is a major step forward. You may disagree. But regardless Agile Programming is the wave of the future. These practices will develop and some of the extreme edges may be worn off, but it's only growing in influence and attraction. Some testers may not like it, but those who don't figure out how to live with it are simply going to be left behind.

Some testers are still upset that they don't have the authority to block the release. Do they think that they now have the authority to block the adoption of these new development methods? They'll need to get on this ship and if they want to try to keep it from the shoals. Stay on the dock if you wish. Bon Voyage!

16. API Testing

Application programmable Interfaces (APIs) are collections of software functions or procedures that can be used by other applications to fulfill their functionality. APIs provide an interface to the software component. These form the critical elements for the developing the applications and are used in varied applications from graph drawing

packages, to speech engines, to web-based airline reservation systems, to computer security components.

Each API is supposed to behave the way it is coded, i.e. it is functionality specific. These APIs may offer different results for different type of the input provided. The errors or the exceptions returned may also vary. However once integrated within a product, the common functionality covers a very minimal code path of the API and the functionality testing / integration testing may cover only those paths. By considering each API as a black box, a generalized approach of testing can be applied. But, there may be some paths which are not tested and lead to bugs in the application. Applications can be viewed and treated as APIs from a testing perspective.

There are some distinctive attributes that make testing of APIs slightly different from testing other common software interfaces like GUI testing.

- ✓ *Testing APIs requires a thorough knowledge of its inner workings* - Some APIs may interact with the OS kernel, other APIs, with other software to offer their functionality. Thus an understanding of the inner workings of the interface would help in analyzing the call sequences and detecting the failures caused.
- ✓ *Adequate programming skills* - API tests are generally in the form of sequences of calls, namely, programs. Each tester must possess expertise in the programming language(s) that are targeted by the API. This would help the tester to review and scrutinize the interface under test when the source code is available.
- ✓ *Lack of Domain knowledge* – Since the testers may not be well trained in using the API, a lot of time might be spent in exploring the interfaces and their usage. This problem can be solved to an extent by involving the testers from the initial stage of development. This would help the testers to have some understanding on the interface and avoid exploring while testing.
- ✓ *No documentation* – Experience has shown that it is hard to create precise and readable documentation. The APIs developed will hardly have any proper documentation available. Without the documentation, it is difficult for the test designer to understand the purpose of calls, the parameter types and possible valid/invalid values, their return values, the calls it makes to other functions,

and usage scenarios. Hence having proper documentation would help test designer design the tests faster.

- ✓ *Access to source code* – The availability of the source code would help tester to understand and analyze the implementation mechanism used; and can identify the loops or vulnerabilities that may cause errors. Thus if the source code is not available then the tester does not have a chance to find anomalies that may exist in the code.
- ✓ *Time constraints* – Thorough testing of APIs is time consuming, requires a learning overhead and resources to develop tools and design tests. Keeping up with deadlines and ship dates may become a nightmare.

Testing of API calls can be done in isolation or in Sequence to vary the order in which the functionality is exercised and to make the API produce useful results from these tests. Designing tests is essentially designing sequences of API calls that have a potential of satisfying the test objectives. This in turn boils down to designing each call with specific parameters and to building a mechanism for handling and evaluating return values.

Thus designing of the test cases can depend on some of the general questions like

- ✓ Which value should a parameter take?
- ✓ What values together make sense?
- ✓ What combination of parameters will make APIs work in a desired manner?
- ✓ What combination will cause a failure, a bad return value, or an anomaly in the operating environment?
- ✓ Which sequences are the best candidates for selection? etc.

Some interesting problems for testers being

1. Ensuring that the test harness varies parameters of the API calls in ways that verify functionality and expose failures. This includes assigning common parameter values as well as exploring boundary conditions.
2. Generating interesting parameter value combinations for calls with two or more parameters.

3. Determining the content under which an API call is made. This might include setting external environment conditions (files, peripheral devices, and so forth) and also internal stored data that affect the API.
4. Sequencing API calls to vary the order in which the functionality is exercised and to make the API produce useful results from successive calls.

By analyzing the problems listed above, a strategy needs to be formulated for testing the API. The API to be tested would require some environment for it to work. Hence it is required that all the conditions and prerequisites understood by the tester. The next step would be to identify and study its points of entry. The GUIs would have items like menus, buttons, check boxes, and combo lists that would trigger the event or action to be taken. Similarly, for APIs, the input parameters, the events that trigger the API would act as the point of entry. Subsequently, a chief task is to analyze the points of entry as well as significant output items. The input parameters should be tested with the valid and invalid values using strategies like the boundary value analysis and equivalence partitioning. The fourth step is to understand the purpose of the routines, the contexts in which they are to be used. Once all this parameter selections and combinations are designed, different call sequences need to be explored.

The steps can be summarized as following

1. Identify the initial conditions required for testing.
2. Identify the parameters – Choosing the values of individual parameters.
3. Identify the combination of parameters – pick out the possible and applicable parameter combinations with multiple parameters.
4. Identify the order to make the calls – deciding the order in which to make the calls to force the API to exhibit its functionality.
5. Observe the output.

1. Identify the initial condition:

The testing of an API would depend largely on the environment in which it is to be tested. Hence initial condition plays a very vital role in understanding and verifying the behavior of the API under test. The initial conditions for testing APIs can be classified as

- ✓ **Mandatory pre-setters.**
- ✓ **Behavioral pre-setters.**

Mandatory Pre-setters

The execution of an API would require some minimal state, environment. These type of initial conditions are classified under the mandatory initialization (Mandatory pre-setters) for the API. For example, a non-static member function API requires an object to be created before it could be called. This is an essential activity required for invoking the API.

Behavioral pre-setters

To test the specific behavior of the API, some additional environmental state is required. These types of initial conditions are called the behavioral pre-setters category of Initial condition. These are optional conditions required by the API and need to be set before invoking the API under test thus influencing its behavior. *Since these influence the behavior of the API under test, they are considered as additional inputs other than the parameters*

Thus to test any API, the environment required should also be clearly understood and set up. Without these criteria, API under test might not function as required and leave the tester's job undone.

2.Input/Parameter Selection: The list of valid input parameters need to be identified to verify that the interface actually performs the tasks that it was designed for. While there is no method that ensures this behavior will be tested completely, using inputs that return quantifiable and verifiable results is the next best thing. The different possible input values (valid and invalid) need to be identified and selected for testing. The techniques like the boundary values analysis and equivalence-partitioning need to be used while trying to consider the input parameter values. The boundary values or the limits that would lead to errors or exceptions need to be identified. It would also be helpful if the data structures and other components that use these data structures apart from the API are analyzed. The data structure can be loaded by using the other components and the API can be tested while the other component is accessing these data structures. Verify that all other dependent components functionality are not affected while the API accesses and manipulates the data structures

The availability of the source code to the testers would help in analyzing the various inputs values that could be possible for testing the API. It would also help in understanding the various paths which could be tested. Therefore, not only are testers required to understand the calls, but also all the constants and data types used by the interface.

3. Identify the combination of parameters: Parameter combinations are extremely important for exercising stored data and computation. In API calls, two independently valid values might cause a fault when used together which might not have occurred with the other combinational values. Therefore, a routine called with two parameters requires selection of values for one based on the value chosen for the other. Often the response of a routine to certain data combinations is incorrectly programmed due to the underlying complex logic.

The API needs to be tested taking into consideration the combination of different parameter. The number of possible combinations of parameters for each call is typically large. For a given set of parameters, if only the boundary values have been selected, the number of combinations, while relatively diminished, may still be prohibitively large. For example, consider an API which takes three parameters as input. The various combinations of different values for the input values and their combinations needs to be identified.

Parameter combination is further complicated by the function overloading capabilities of many modern programming languages. It is important to isolate the differences between such functions and take into account that their use is context driven. The APIs can also be tested to check that there are no memory leaks after they are called. This can be verified by continuously calling the API and observing the memory utilization.

4.Call Sequencing: When combinations of possible arguments to each individual call are unmanageable, the number of possible call sequences is infinite. Parameter selection and combination issues further complicate the problem call-sequencing problem. Faults caused by improper call sequences tend to give rise to some of the most dangerous problems in software. Most security vulnerabilities are caused by the execution of some such seemingly improbable sequences.

5.Observe the output: The outcome of an execution of an API depends upon the behavior of that API, the test condition and the environment. The outcome of an API can

be at different ways i.e., some could generally return certain data or status but for some of the API's, it might not return or shall be just waiting for a period of time, triggering another event, modifying certain resource and so on.

The tester should be aware of the output that needs to be expected for the API under test. The outputs returned for various input values like valid/invalid, boundary values etc needs to be observed and analyzed to validate if they are as per the functionality. All the error codes returned and exceptions returned for all the input combinations should be evaluated.

API Testing Tools: There are many testing tools available. Depending on the level of testing required, different tools could be used. Some of the API testing tools available are mentioned here.

JVerify: This is from Man Machine Systems.

JVerify is a Java class/API testing tool that supports a unique invasive testing model. The invasive model allows access to the internals (private elements) of any Java object from within a test script. The ability to invade class internals facilitates more effective testing at class level, since controllability and observability are enhanced. This can be very valuable when a class has not been designed for testability.

JavaSpec: JavaSpec is a SunTest's API testing tool. It can be used to test Java applications and libraries through their API. JavaSpec guides the users through the entire test creation process and lets them focus on the most critical aspects of testing. Once the user has entered the test data and assertions, JavaSpec automatically generates self-checking tests, HTML test documentation, and detailed test reports.

Here is an example of how to automate the API testing.

Assumptions: -

1. Test engineer is supposed to test some API.
2. The API's are available in form of library (.lib).
3. Test engineer has the API document.

There are mainly two things to test in API testing: -

1. Black box testing of the API's
2. Interaction / integration testing of the API's.

By black box testing of the API mean that we have to test the API for outputs. In simple words when we give a known input (parameters to the API) then we also know the ideal output. So we have to check for the actual out put against the idle output.

For this we can write a simple c program that will do the following: -

- a) Take the parameters from a text file (this file will contain many of such input parameters).
- b) Call the API with these parameters.
- c) Match the actual and idle output and also check the parameters for good values that are passed with reference (pointers).
- d) Log the result.

Secondly we have test the integration of the API's.

For example there are two API's say

Handle h = handle createContext (void);

When the handle to the device is to be closed then the corresponding function

Bool bishandledeleted = bool deletecontext (handle &h);

Here we have to call the two API's and check if they are handled by the created createContext () and are deleted by the deletecontext ().

This will ensure that these two API's are working fine.

For this we can write a simple c program that will do the following: -

- a) Call the two API's in the same order.
- b) Pass the output parameter of the first as the input of the second
- c) Check for the output parameter of the second API
- d) Log the result.

The example is over simplified but this works because we are using this kind of test tool for extensive regression testing of our API library.

17. Understanding Rapid Testing

Rapid testing is the testing software faster than usual, without compromising on the standards of quality. It is the technique to test as thorough as reasonable within the constraints. This technique looks at testing as a process of heuristic inquiry and logically speaking it should be based on exploratory testing techniques.

Although most projects undergo continuous testing, it does not usually produce the

<http://www.SofTReL.org>

information required to deal with the situations where it is necessary to make an instantaneous assessment of the product's quality at a particular moment. In most cases the testing is scheduled for just prior to launch and conventional testing techniques often cannot be applied to software that is incomplete or subject to constant change. At times like these Rapid Testing can be used.

It can be said that rapid testing has a structure that is built on a foundation of four components namely,

- People
- Integrated test process
- Static Testing and
- Dynamic Testing

There is a need for people who can handle the pressure of tight schedules. They need to be productive contributors even through the early phases of the development life cycle. According to James Bach, a core skill is the ability to think critically.

It should also be noted that dynamic testing lies at the heart of the software testing process, and the planning, design, development, and execution of dynamic tests should be performed well for any testing process to be efficient.

THE RAPID TESTING PRACTICE

It would help us if we scrutinize each phase of a development process to see how the efficiency, speed and quality of testing can be improved, bearing in mind the following factors:

- Actions that the test team can take to prevent defects from escaping. For example, practices like extreme programming and exploratory testing.
- Actions that the test team can take to manage risk to the development schedule.
- The information that can be obtained from each phase so that the test team can speed up the activities.

If a test process is designed around the answers to these questions, both the speed of testing and the quality of the final product should be enhanced.

Some of the aspects that can be used while rapid testing are given below:

1. Test for link integrity
2. Test for disabled accessibility
3. Test the default settings

4. Check the navigation's
5. Check for input constraints by injecting special characters at the sources of data
6. Run Multiple instances
7. Check for interdependencies and stress them
8. Test for consistency of design
9. Test for compatibility
10. Test for usability
11. Check for the possible variability's and attack them
12. Go for possible stress and load tests
13. And our favorite – banging the keyboard

18. Test Ware Development

Test Ware development is the key role of the Testing Team. What comprises Test Ware and some guidelines to build the test ware is discussed below:

18.1 Test Strategy

Before starting any testing activities, the team lead will have to think a lot & arrive at a strategy. This will describe the approach, which is to be adopted for carrying out test activities including the planning activities. This is a formal document and the very first document regarding the testing area and is prepared at a very early stag in SDLC. This document must provide generic test approach as well as specific details regarding the project. The following areas are addressed in the test strategy document.

18.1.1 Test Levels

The test strategy must talk about what are the test levels that will be carried out for that particular project. Unit, Integration & System testing will be carried out in all projects. But many times, the integration & system testing may be combined. Details like this may be addressed in this section.

18.1.2 Roles and Responsibilities

The roles and responsibilities of test leader, individual testers, project manager are to be clearly defined at a project level in this section. This may not have names associated: but the role has to be very clearly defined. The review and approval mechanism must be stated here for test plans and other test documents. Also, we have to state who reviews

the test cases, test records and who approved them. The documents may go thru a series of reviews or multiple approvals and they have to be mentioned here.

18.1.3 Testing Tools

Any testing tools, which are to be used in different test levels, must be, clearly identified. This includes justifications for the tools being used in that particular level also.

18.1.4 Risks and Mitigation

Any risks that will affect the testing process must be listed along with the mitigation. By documenting the risks in this document, we can anticipate the occurrence of it well ahead of time and then we can proactively prevent it from occurring. Sample risks are dependency of completion of coding, which is done by sub-contractors, capability of testing tools etc.

18.1.5 Regression Test Approach

When a particular problem is identified, the programs will be debugged and the fix will be done to the program. To make sure that the fix works, the program will be tested again for that criteria. Regression test will make sure that one fix does not create some other problems in that program or in any other interface. So, a set of related test cases may have to be repeated again, to make sure that nothing else is affected by a particular fix. How this is going to be carried out must be elaborated in this section. In some companies, whenever there is a fix in one unit, all unit test cases for that unit will be repeated, to achieve a higher level of quality.

18.1.6 Test Groups

From the list of requirements, we can identify related areas, whose functionality is similar. These areas are the test groups. For example, in a railway reservation system, anything related to ticket booking is a functional group; anything related with report generation is a functional group. Same way, we have to identify the test groups based on the functionality aspect.

18.1.7 Test Priorities

Among test cases, we need to establish priorities. While testing software projects, certain test cases will be treated as the most important ones and if they fail, the product cannot be released. Some other test cases may be treated like cosmetic and if they fail, we can release the product without much compromise on the functionality.

This priority levels must be clearly stated. These may be mapped to the test groups also.

18.1.8 Test Status Collections and Reporting

When test cases are executed, the test leader and the project manager must know, where exactly we stand in terms of testing activities. To know where we stand, the inputs from the individual testers must come to the test leader. This will include, what test cases are executed, how long it took, how many test cases passed and how many-failed etc. Also, how often we collect the status is to be clearly mentioned. Some companies will have a practice of collecting the status on a daily basis or weekly basis. This has to be mentioned clearly.

18.1.9 Test Records Maintenance

When the test cases are executed, we need to keep track of the execution details like when it is executed, who did it, how long it took, what is the result etc. This data must be available to the test leader and the project manager, along with all the team members, in a central location. This may be stored in a specific directory in a central server and the document must say clearly about the locations and the directories. The naming convention for the documents and files must also be mentioned.

18.1.10 Requirements Traceability Matrix

Ideally each software developed must satisfy the set of requirements completely. So, right from design, each requirement must be addressed in every single document in the software process. The documents include the HLD, LLD, source codes, unit test cases, integration test cases and the system test cases. Refer the following sample table which describes Requirements Traceability Matrix process. In this matrix, the rows will have the requirements. For every document {HLD, LLD etc}, there will be a separate column. So, in every cell, we need to state, what section in HLD addresses a particular requirement. Ideally, if every requirement is addressed in every single document, all the individual cells must have valid section ids or names filled in. Then we know that every requirement is addressed. In case of any missing of requirement, we need to go back to the document and correct it, so that it addressed the requirement.

For testing at each level, we may have to address the requirements. One integration and the system test case may address multiple requirements.

	DTP Scenario No	DTC Id	Code	LLD Section
Requirement 1	+ve/-ve	1,2,3,4		
Requirement 2	+ve/-ve	1,2,3,4		
Requirement 3	+ve/-ve	1,2,3,4		
Requirement 4	+ve/-ve	1,2,3,4		
...				
...				
...				
...				
...				
...				
...				
Requirement N	+ve/-ve	1,2,3,4		
	TESTER	TESTER	DEVELOPER	TEST LEAD

18.1.11 Test Summary

The senior management may like to have test summary on a weekly or monthly basis. If the project is very critical, they may need it on a daily basis also. This section must address what kind of test summary reports will be produced for the senior management along with the frequency.

The test strategy must give a clear vision of what the testing team will do for the whole project for the entire duration. This document will/may be presented to the client also, if needed. The person, who prepares this document, must be functionally strong in the product domain, with a very good experience, as this is the document that is going to drive the entire team for the testing activities. Test strategy must be clearly explained to the testing team members tight at the beginning of the project.

18.2 Test Plan

The test strategy identifies multiple test levels, which are going to be performed for the project. Activities at each level must be planned well in advance and it has to be formally documented. Based on the individual plans only, the individual test levels are carried out.

The plans are to be prepared by experienced people only. In all test plans, the ETVX {Entry-Task-Validation-Exit} criteria are to be mentioned. Entry means the entry point

to that phase. For example, for unit testing, the coding must be complete and then only one can start unit testing. Task is the activity that is performed. Validation is the way in which the progress and correctness and compliance are verified for that phase. Exit tells the completion criteria of that phase, after the validation is done. For example, the exit criterion for unit testing is all unit test cases must pass.

ETVX is a modeling technique for developing worldly and atomic level models. It stands for Entry, Task, Verification and Exit. It is a task-based model where the details of each task are explicitly defined in a specification table against each phase i.e. Entry, Exit, Task, Feedback In, Feedback Out, and measures.

There are two types of cells, unit cells and implementation cells. The implementation cells are basically unit cells containing the further tasks.

For example if there is a task of size estimation, then there will be a unit cell of size estimation. Then since this task has further tasks namely, define measures, estimate size. The unit cell containing these further tasks will be referred to as the implementation cell and a separate table will be constructed for it.

A purpose is also stated and the viewer of the model may also be defined e.g. top management or customer.

18.2.1 Unit Test Plan {UTP}

The unit test plan is the overall plan to carry out the unit test activities. The lead tester prepares it and it will be distributed to the individual testers, which contains the following sections.

18.2.1.1 What is to be tested?

The unit test plan must clearly specify the scope of unit testing. In this, normally the basic input/output of the units along with their basic functionality will be tested. In this case mostly the input units will be tested for the format, alignment, accuracy and the totals. The UTP will clearly give the rules of what data types are present in the system, their format and their boundary conditions. This list may not be exhaustive; but it is better to have a complete list of these details.

18.2.1.2 Sequence of Testing

The sequences of test activities that are to be carried out in this phase are to be listed in this section. This includes, whether to execute positive test cases first or negative test cases first, to execute test cases based on the priority, to execute test cases based on test groups etc. Positive test cases prove that the system performs what is supposed to

do; negative test cases prove that the system does not perform what is not supposed to do. Testing the screens, files, database etc., are to be given in proper sequence.

18.2.1.4 Basic Functionality of Units

How the independent functionalities of the units are tested which excludes any communication between the unit and other units. The interface part is out of scope of this test level. Apart from the above sections, the following sections are addressed, very specific to unit testing.

- Unit Testing Tools
- Priority of Program units
- Naming convention for test cases
- Status reporting mechanism
- Regression test approach
- ETVX criteria

18.2.2 Integration Test Plan

The integration test plan is the overall plan for carrying out the activities in the integration test level, which contains the following sections.

2.2.1 What is to be tested?

This section clearly specifies the kinds of interfaces fall under the scope of testing internal, external interfaces, with request and response is to be explained. This need not go deep in terms of technical details but the general approach how the interfaces are triggered is explained.

18.2.2.1 Sequence of Integration

When there are multiple modules present in an application, the sequence in which they are to be integrated will be specified in this section. In this, the dependencies between the modules play a vital role. If a unit B has to be executed, it may need the data that is fed by unit A and unit X. In this case, the units A and X have to be integrated and then using that data, the unit B has to be tested. This has to be stated to the whole set of units in the program. Given this correctly, the testing activities will lead to the product, slowly building the product, unit by unit and then integrating them.

18.2.2.2 List of Modules and Interface Functions

There may be N number of units in the application, but the units that are going to communicate with each other, alone are tested in this phase. If the units are designed in such a way that they are mutually independent, then the interfaces do not come into picture. This is almost impossible in any system, as the units have to communicate to other units, in order to get different types of functionalities executed. In this section, we need to list the units and for what purpose it talks to the others need to be mentioned. This will not go into technical aspects, but at a higher level, this has to be explained in plain English.

Apart from the above sections, the following sections are addressed, very specific to integration testing.

- Integration Testing Tools
- Priority of Program interfaces
- Naming convention for test cases
- Status reporting mechanism
- Regression test approach
- ETVX criteria
- Build/Refresh criteria {When multiple programs or objects are to be linked to arrived at single product, and one unit has some modifications, then it may need to rebuild the entire product and then load it into the integration test environment. When and how often, the product is rebuilt and refreshed is to be mentioned}.

18.2.3 System Test Plan {STP}

The system test plan is the overall plan carrying out the system test level activities. In the system test, apart from testing the functional aspects of the system, there are some special testing activities carried out, such as stress testing etc. The following are the sections normally present in system test plan.

18.2.3.1 What is to be tested?

This section defines the scope of system testing, very specific to the project. Normally, the system testing is based on the requirements. All requirements are to be verified in the scope of system testing. This covers the functionality of the product. Apart from this what special testing is performed are also stated here.

18.2.3.2 Functional Groups and the Sequence

The requirements can be grouped in terms of the functionality. Based on this, there may be priorities also among the functional groups. For example, in a banking application, anything related to customer accounts can be grouped into one area, anything related to inter-branch transactions may be grouped into one area etc. Same way for the product being tested, these areas are to be mentioned here and the suggested sequences of testing of these areas, based on the priorities are to be described.

18.2.3.3 Special Testing Methods

This covers the different special tests like load/volume testing, stress testing, interoperability testing etc. These testing are to be done based on the nature of the product and it is not mandatory that every one of these special tests must be performed for every product.

Apart from the above sections, the following sections are addressed, very specific to system testing.

- System Testing Tools
- Priority of functional groups
- Naming convention for test cases
- Status reporting mechanism
- Regression test approach
- ETVX criteria
- Build/Refresh criteria

18.2.4 Acceptance Test Plan {ATP}

The client at their place performs the acceptance testing. It will be very similar to the system test performed by the Software Development Unit. Since the client is the one who decides the format and testing methods as part of acceptance testing, there is no specific clue on the way they will carry out the testing. But it will not differ much from the system testing. Assume that all the rules, which are applicable to system test, can be implemented to acceptance testing also.

Since this is just one level of testing done by the client for the overall product, it may include test cases including the unit and integration test level details.

A sample Test Plan Outline along with their description is as shown below:

<http://www.SofTReL.org>

Test Plan Outline

1. BACKGROUND – This item summarizes the functions of the application system and the tests to be performed.
2. INTRODUCTION
3. ASSUMPTIONS – Indicates any anticipated assumptions which will be made while testing the application.
4. TEST ITEMS - List each of the items (programs) to be tested.
5. FEATURES TO BE TESTED - List each of the features (functions or requirements) which will be tested or demonstrated by the test.
6. FEATURES NOT TO BE TESTED - Explicitly lists each feature, function, or requirement which won't be tested and why not.
7. APPROACH - Describe the data flows and test philosophy.
Simulation or Live execution, Etc. This section also mentions all the approaches which will be followed at the various stages of the test execution.
8. ITEM PASS/FAIL CRITERIA Blanket statement - Itemized list of expected output and tolerances
9. SUSPENSION/RESUMPTION CRITERIA - Must the test run from start to completion?
Under what circumstances it may be resumed in the middle?
Establish check-points in long tests.
10. TEST DELIVERABLES - What, besides software, will be delivered?
Test report
Test software
11. TESTING TASKS Functional tasks (e.g., equipment set up)
Administrative tasks
12. ENVIRONMENTAL NEEDS
Security clearance
Office space & equipment
Hardware/software requirements

13. RESPONSIBILITIES

Who does the tasks in Section 10?

What does the user do?

14. STAFFING & TRAINING

15. SCHEDULE

16. RESOURCES

17. RISKS & CONTINGENCIES

18. APPROVALS

The schedule details of the various test pass such as Unit tests, Integration tests, System Tests should be clearly mentioned along with the estimated efforts.

18.3 Test Case Documents

Designing good test cases is a complex art. The complexity comes from three sources:

- Test cases help us discover information. Different types of tests are more effective for different classes of information.
- Test cases can be “good” in a variety of ways. No test case will be good in all of them.
- People tend to create test cases according to certain testing styles, such as domain testing or risk-based testing. Good domain tests are different from good risk-based tests.

What’s a test case?

“A test case specifies the pretest state of the IUT and its environment, the test inputs or conditions, and the expected result. The expected result specifies what the IUT should produce from the test inputs. This specification includes messages generated by the IUT, exceptions, returned values, and resultant state of the IUT and its environment. Test cases may also specify initial and resulting conditions for other objects that constitute the IUT and its environment.”

What’s a scenario?

A *scenario* is a hypothetical story, used to help a person think through a complex problem or system.

Characteristics of Good Scenarios

A scenario test has five key characteristics. It is (a) a story that is (b) motivating, (c) credible, (d) complex, and (e) easy to evaluate.

The primary objective of test case design is to derive a set of tests that have the highest attitude of discovering defects in the software. Test cases are designed based on the analysis of requirements, use cases, and technical specifications, and they should be developed in parallel with the software development effort.

A test case describes a set of actions to be performed and the results that are expected. A test case should target specific functionality or aim to exercise a valid path through a use case. This should include invalid user actions and illegal inputs that are not necessarily listed in the use case. A test case is described depends on several factors, e.g. the number of test cases, the frequency with which they change, the level of automation employed, the skill of the testers, the selected testing methodology, staff turnover, and risk.

The test cases will have a generic format as below.

Test case ID - The test case id must be unique across the application

Test case description - The test case description must be very brief.

Test prerequisite - The test pre-requisite clearly describes what should be present in the system, before the test can be executes.

Test Inputs - The test input is nothing but the test data that is prepared to be fed to the system.

Test steps - The test steps are the step-by-step instructions on how to carry out the test.

Expected Results - The expected results are the ones that say what the system must give as output or how the system must react based on the test steps.

Actual Results – The actual results are the ones that say outputs of the action for the given inputs or how the system reacts for the given inputs.

Pass/Fail - If the Expected and Actual results are same then test is Pass otherwise Fail.

The test cases are classified into positive and negative test cases. Positive test cases are designed to prove that the system accepts the valid inputs and then process them correctly. Suitable techniques to design the positive test cases are Specification derived tests, Equivalence partitioning and State-transition testing. The negative test cases are

designed to prove that the system rejects invalid inputs and does not process them. Suitable techniques to design the negative test cases are Error guessing, Boundary value analysis, internal boundary value testing and State-transition testing. The test cases details must be very clearly specified, so that a new person can go through the test cases step and step and is able to execute it. The test cases will be explained with specific examples in the following section.

For example consider online shopping application. At the user interface level the client request the web server to display the product details by giving email id and Username. The web server processes the request and will give the response. For this application we will design the unit, Integration and system test cases.

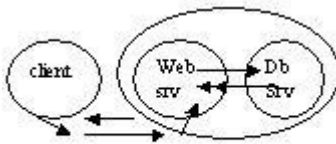


Figure 6.Web based application

Unit Test Cases (UTC)

These are very specific to a particular unit. The basic functionality of the unit is to be understood based on the requirements and the design documents. Generally, Design document will provide a lot of information about the functionality of a unit. The Design document has to be referred before UTC is written, because it provides the actual functionality of how the system must behave, for given inputs.

For example, In the Online shopping application, If the user enters valid Email id and Username values, let us assume that Design document says, that the system must display a product details and should insert the Email id and Username in database table. If user enters invalid values the system will display appropriate error message and will not store it in database.

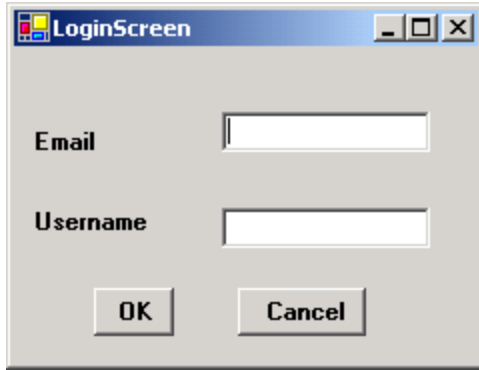


Figure 7: Snapshot of Login Screen

Test Conditions for the fields in the Login screen

Email-It should be in this format (For Eg clickme@yahoo.com).

Username – It should accept only alphabets not greater than 6.Numerics and special type of characters are not allowed.

Test Prerequisite: The user should have access to Customer Login screen form screen

Negative Test Case

Project Name-Online shopping

Version-1.1

Module-Catalog

T est #	Description	Test Inputs	Expected Results	Actual results	Pass/Fail
1	Check for inputting values in Email field	Email=keerthi@rediffmail Username=Xavier	Inputs should not be accepted. It should display message “Enter valid Email”		
2	Check for inputting values in Email field	Email=john26#rediffmail.com Username=John	Inputs should not be accepted. It should display message “Enter valid Email”		
3	Check for inputting values in Username field	Email=shilpa@yahoo.com Username=Mark24	Inputs should not be accepted. It should display		

			message “Enter correct Username”		
--	--	--	----------------------------------	--	--

Positive Test Case

Test #	Description	Test Inputs	Expected Results	Actual results	Pass/Fail
1	Check for inputting values in Email field	Email=shan@yahoo.com Username=dave	Inputs should be accepted.		
2	Check for inputting values in Email field	Email=knki@rediffmail.co m Username=john	Inputs should be accepted.		
3	Check for inputting values in Username field	Email=xav@yahoo.com Username=mark	Inputs should be accepted.		

Integration Test Cases

Before designing the integration test cases the testers should go through the Integration test plan. It will give complete idea of how to write integration test cases. The main aim of integration test cases is that it tests the multiple modules together. By executing these test cases the user can find out the errors in the interfaces between the Modules.

For example, in online shopping, there will be Catalog and Administration module. In catalog section the customer can track the list of products and can buy the products online. In administration module the admin can enter the product name and information related to it.

Table3: Integration Test Cases

Test #	Description	Test Inputs	Expected Results	Actual results	Pass/Fail
--------	-------------	-------------	------------------	----------------	-----------

1	Check for Login Screen	Enter values in Email and UserName. For Eg: Email =shilpa@yahoo.com Username=shilpa	Inputs should be accepted.		
	Backend Verification	Select email, username from Cus;	The entered Email and Username should be displayed at sqlprompt.		
2	Check for Product Information	Click product information link	It should display complete details of the product		
3	Check for admin screen	Enter values in Product Id and Product name fields. For Eg: Product Id-245 Product name-Norton Antivirus	Inputs should be accepted.		
	Backend verification	Select pid , pname from Product;	The entered Product id and Product name should be displayed at the sql prompt.		

NOTE: The tester has to execute above unit and Integration test cases after coding. And He/She has to fill the actual results and Pass/fail columns. If the test cases fail then defect report should be prepared.

System Test Cases: -

The system test cases meant to test the system as per the requirements; end-to end. This is basically to make sure that the application works as per SRS. In system test cases, (generally in system testing itself), the testers are supposed to act as an end user.

<http://www.SofTReL.org>

user. So, system test cases normally do concentrate on the functionality of the system, inputs are fed through the system and each and every check is performed using the system itself. Normally, the verifications done by checking the database tables directly or running programs manually are not encouraged in the system test.

The system test must focus on functional groups, rather than identifying the program units. When it comes to system testing, it is assumed that the interfaces between the modules are working fine (integration passed).

Ideally the test cases are nothing but a union of the functionalities tested in the unit testing and the integration testing. Instead of testing the system inputs outputs through database or external programs, everything is tested through the system itself. For example, in an online shopping application, the catalog and administration screens (program units) would have been independently unit tested and the test results would be verified through the database. In system testing, the tester will mimic as an end user and hence checks the application through its output.

There are occasions, where some/many of the integration and unit test cases are repeated in system testing also; especially when the units are tested with test stubs before and not actually tested with other real modules, during system testing those cases will be performed again with real modules/data in

19. Defect Management

Defects determine the effectiveness of the Testing what we do. If there are no defects, it directly implies that we don't have our job. There are two points worth considering here, either the developer is so strong that there are no defects arising out, or the test engineer is weak. In many situations, the second is proving correct. This implies that we lack the knack. In this section, let us understand Defects.

19.1 What is a Defect?

For a test engineer, a defect is following: -

- Any deviation from specification
- Anything that causes user dissatisfaction
- Incorrect output
- Software does not do what it intended to do.

Bug / Defect / Error: -

- Software is said to have **bug** if it features deviates from specifications.
- Software is said to have **defect** if it has unwanted side effects.

- Software is said to have **Error** if it gives incorrect output.

But as for a test engineer all are same as the above definition is only for the purpose of documentation or indicative.

19.2 Defect Taxonomies

Categories of Defects:

All software defects can be broadly categorized into the below mentioned types:

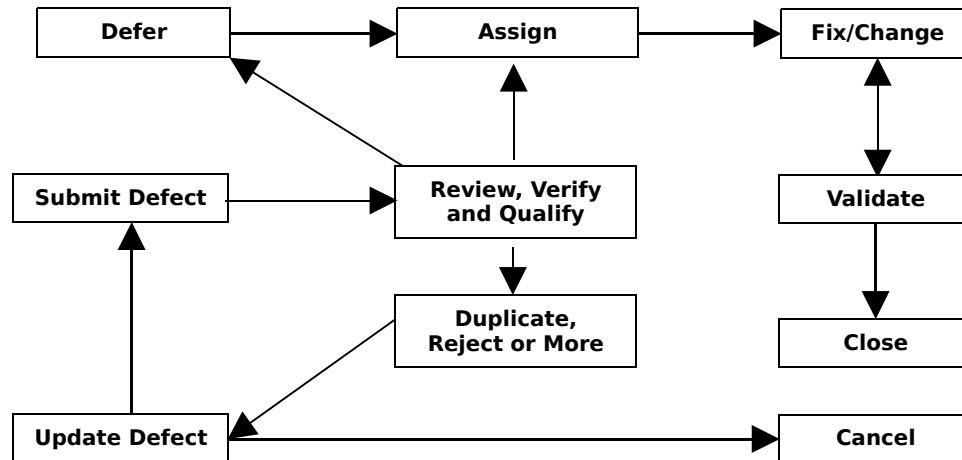
- Errors of commission: something wrong is done
- Errors of omission: something left out by accident
- Errors of clarity and ambiguity: different interpretations
- Errors of speed and capacity

However, the above is a broad categorization; below we have for you a host of varied types of defects that can be identified in different software applications:

1. Conceptual bugs / Design bugs
2. Coding bugs
3. Integration bugs
4. User Interface Errors
5. Functionality
6. Communication
7. Command Structure
8. Missing Commands
9. Performance
10. Output
11. Error Handling Errors
12. Boundary-Related Errors
13. Calculation Errors
14. Initial and Later States
15. Control Flow Errors
16. Errors in Handling Data
17. Race Conditions Errors
18. Load Conditions Errors
19. Hardware Errors
20. Source and Version Control Errors
21. Documentation Errors
22. Testing Errors

19.3 Life Cycle of a Defect

The following self explanatory figure explains the life cycle of a defect:



20. Metrics for Testing

What is a Metric?

'Metric' is a measure to quantify software, software development resources, and/or the software development process. A Metric can quantify any of the following factors:

- Schedule,
- Work Effort,
- Product Size,
- Project Status, and
- Quality Performance

Measuring enables....

Metrics enables estimation of future work.

That is, considering the case of testing - Deciding the product is fit for shipment or delivery depends on the rate the defects are found and fixed. Defect collected and fixed is one kind of metric. (www.processimpact.com)

As defined in the MISRA Report,

It is beneficial to classify metrics according to their usage. IEEE 928.1 [4] identifies two classes:

- i) Process – Activities performed in the production of the Software

- ii) Product – An output of the Process, for example the software or its documentation.

Defects are analyzed to identify which are the major causes of defect and which is the phase that introduces most defects. This can be achieved by performing Pareto analysis of defect causes and defect introduction phases. The main requirements for any of these analysis is Software Defect Metrics.

Few of the Defect Metrics are:

Defect Density: (No. Of Defects Reported by SQA + No. Defects Reported By Peer Review)/Actual Size.

The Size can be in KLOC, SLOC, or Function Points. The method used in the Organization to measure the size of the Software Product.

The SQA is considered to be the part of the Software testing team.

Test effectiveness: $t / (t + Uat)$ where t = total no. of defects reported during testing and Uat = total no. of defects reported during User acceptance testing

User Acceptance Testing is generally carried out using the Acceptance Test Criteria according to the Acceptance Test Plan.

Defect Removal Efficiency:

(Total No Of Defects Removed / Total No. Of Defects Injected)*100 at various stages of SDLC

Description

This metric will indicate the effectiveness of the defect identification and removal in stages for a given project

Formula

- Requirements: $DRE = [(Requirement\ defects\ corrected\ during\ Requirements\ phase) / (Requirement\ defects\ injected\ during\ Requirements\ phase)] * 100$
- Design: $DRE = [(Design\ defects\ corrected\ during\ Design\ phase) / (Defects\ identified\ during\ Requirements\ phase + Defects\ injected\ during\ Design\ phase)] * 100$
- Code: $DRE = [(Code\ defects\ corrected\ during\ Coding\ phase) / (Defects\ identified\ during\ Requirements\ phase + Defects\ identified\ during\ Design\ phase + Defects\ injected\ during\ coding\ phase)] * 100$

- Overall: $DRE = [(Total\ defects\ corrected\ at\ all\ phases\ before\ delivery) / (Total\ defects\ detected\ at\ all\ phases\ before\ and\ after\ delivery)] * 100$

Metric Representation

Percentage

Calculated at

Stage completion or Project Completion

Calculated from

Bug Reports and Peer Review Reports

Defect Distribution: Percentage of Total defects Distributed across Requirements Analysis, Design Reviews, Code Reviews, Unit Tests, Integration Tests, System Tests, User Acceptance Tests, Review by Project Leads and Project Managers.

Software Process Metrics are measures which provide information about the performance of the development process itself.

Purpose:

1. Provide an Indicator to the Ultimate Quality of Software being Produced
2. Assists to the Organization to improve its development process by Highlighting areas of Inefficiency or error-prone areas of the process.

Software Product Metrics are measures of some attribute of the Software Product. (Example, Source Code).

Purpose:

1. Used to assess the quality of the output

What are the most general metrics?**Requirements Management****Metrics Collected**

1. Requirements by state – Accepted, Rejected, Postponed
2. No. of baselined requirements
3. Number of requirements modified after base lining

Derived Metrics

1. Requirements Stability Index (RSI)
2. Requirements to Design Traceability

Project Management

Metrics Collected	Derived Metrics
1. Planned No. of days	1. Schedule Variance
2. Actual No. of days	

1. Estimated effort	1. Effort Variance
2. Actual Effort	
1. Estimated Cost	1. Cost Variance
2. Actual Cost	
1. Estimated Size	1. Size Variance
2. Actual Size	

Testing & Review**Metrics Collected**

1. No. of defects found by Reviews
2. No. of defects found by Testing
3. No. of defects found by Client
4. Total No. of defects found by Reviews

Derived Metrics

1. Overall Review Effectiveness (ORE)
2. Overall Test Effectiveness

Peer Reviews**Metrics Collected**

1. KLOC / FP per person hour (Language) for Preparation
2. KLOC / FP per person hour (Language) for Review Meeting
3. No. of pages / hour reviewed during preparation
4. Average number of defects found by Reviewer during Preparation
5. No. of pages / hour reviewed during Review Meeting
6. Average number of defects found by Reviewer during Review Meeting
7. Review Team Size Vs Defects
8. Review speed Vs Defects
9. Major defects found during Review Meeting
10. Defects Vs Review Effort

Derived Metrics

1. Review Effectiveness (Major)
2. Total number of defects found by reviews for a project

Other Metrics**Metrics Collected**

1. No. of Requirements Designed
2. No. of Requirements not Designed
3. No. of Design elements matching Requirements
4. No. of Design elements not matching Requirements
5. No. of Requirements Tested

6. No. of Requirements not Tested
7. No. of Test Cases with matching Requirements
8. No. of Test Cases without matching Requirements
9. No. of Defects by Severity
10. No. of Defects by stage of - Origin, Detection, Removal

Derived Metrics

1. Defect Density
2. No. of Requirements Designed Vs not Designed
3. No. of Requirements Tested Vs not Tested
4. Defect Removal Efficiency (DRE)

Some Metrics Explained

Schedule Variance (SV)

Description

This metric gives the variation of actual schedule vs. the planned schedule. This is calculated for each project – stage wise

Formula

$$SV = [(Actual\ no.\ of\ days - Planned\ no.\ of\ days) / Planned\ no.\ of\ days] * 100$$

Metric Representation

Percentage

Calculated at

Stage completion

Calculated from

Software Project Plan for planned number of days for completing each stage and for actual number of days taken to complete each stage

Defect Removal Efficiency (DRE)

Description

This metric will indicate the effectiveness of the defect identification and removal in stages for a given project

Formula

- Requirements: $DRE = [(Requirement\ defects\ corrected\ during\ Requirements\ phase) / (Requirement\ defects\ injected\ during\ Requirements\ phase)] * 100$
- Design: $DRE = [(Design\ defects\ corrected\ during\ Design\ phase) / (Defects\ identified\ during\ Requirements\ phase + Defects\ injected\ during\ Design\ phase)] * 100$
- Code: $DRE = [(Code\ defects\ corrected\ during\ Coding\ phase) / (Defects\ identified\ during\ Requirements\ phase + Defects\ identified\ during\ Design\ phase + Defects\ injected\ during\ coding\ phase)] * 100$

- Overall: $DRE = [(Total\ defects\ corrected\ at\ all\ phases\ before\ delivery) / (Total\ defects\ detected\ at\ all\ phases\ before\ and\ after\ delivery)] * 100$

Metric Representation

Percentage

Calculated at

Stage completion or Project Completion

Calculated from

Bug Reports and Peer Review Reports

Overall Review Effectiveness**Description**

This metric will indicate the effectiveness of the Review process in identifying the defects for a given project

Formula

- Overall Review Effectiveness: $ORE = [(Number\ of\ defects\ found\ by\ reviews) / (Total\ number\ of\ defects\ found\ by\ reviews + Number\ of\ defects\ found\ during\ Testing + Number\ of\ defects\ found\ during\ post-delivery)] * 100$

Metric Representation

- Percentage

Calculated at

- Monthly
- Stage completion or Project Completion

Calculated from

- Peer reviews, Formal Reviews
- Test Reports
- Customer Identified Defects

Overall Test Effectiveness (OTE)**Description**

This metric will indicate the effectiveness of the Testing process in identifying the defects for a given project during the testing stage

Formula

- Overall Test Effectiveness: $OTE = [(Number\ of\ defects\ found\ during\ testing) / (Total\ number\ of\ defects\ found\ during\ Testing + Number\ of\ defects\ found\ during\ post\ delivery)] * 100$

Metric Representation

- Percentage

Calculated at

- Monthly
- Build completion or Project Completion

Calculated from

- Test Reports
- Customer Identified Defects

Effort Variance (EV)**Description**

This metric gives the variation of actual effort vs. the estimated effort. This is calculated for each project Stage wise

Formula

- $EV = [(Actual\ person\ hours - Estimated\ person\ hours) / Estimated\ person\ hours] * 100$

Metric Representation

- Percentage

Calculated at

- Stage completion as identified in SPP

Calculated from

- Estimation sheets for estimated values in person hours, for each activity within a given stage and Actual Worked Hours values in person hours.

Cost Variance (CV)**Description**

This metric gives the variation of actual cost Vs the estimated cost. This is calculated for each project Stage wise

Formula

- $CV = [(Actual\ Cost - Estimated\ Cost) / Estimated\ Cost] * 100$

Metric Representation

- Percentage

Calculated at

- Stage completion

Calculated from

- Estimation sheets for estimated values in dollars or rupees, for each activity within a given stage
- Actual cost incurred

Size Variance

Description

This metric gives the variation of actual size Vs. the estimated size. This is calculated for each project stage wise

Formula

- $\text{Size Variance} = [(\text{Actual Size} - \text{Estimated Size}) / \text{Estimated Size}] * 100$

Metric Representation

- Percentage

Calculated at

- Stage completion
- Project Completion

Calculated from

- Estimation sheets for estimated values in Function Points or KLOC
- Actual size

Productivity on Review Preparation – Technical**Description**

This metric will indicate the effort spent on preparation for Review. Use this to calculate for languages used in the Project

Formula

For every language (such as C, C++, Java, XSL, etc...) used, calculate

- $(\text{KLOC or FP}) / \text{hour} (* \text{Language})$

*Language – C, C++, Java, XML, etc...

Metric Representation

- KLOC or FP per hour

Calculated at

- Monthly
- Build completion

Calculated from

- Peer Review Report

Number of defects found per Review Meeting**Description**

This metric will indicate the number of defects found during the Review Meeting across various stages of the Project

Formula

- Number of defects per Review Meeting

Metric Representation

- Defects / Review Meeting

Calculated at

- Monthly
- Completion of Review

Calculated from

- Peer Review Report
- Peer Review Defect List

Review Team Efficiency (Review Team Size Vs Defects Trend)**Description**

This metric will indicate the Review Team size and the defects trend. This will help to determine the efficiency of the Review Team

Formula

- Review Team Size to the Defects trend

Metric Representation

- Ratio

Calculated at

- Monthly
- Completion of Review

Calculated from

- Peer Review Report
- Peer Review Defect List

Review Effectiveness**Description**

This metric will indicate the effectiveness of the Review process

Formula

Review Effectiveness = $\left[\frac{\text{Number of defects found by Reviews}}{(\text{Total number of defects found by reviews}) + \text{Testing}} \right] * 100$

Metric Representation

- Percentage

Calculated at

- Completion of Review or Completion of Testing stage

Calculated from

- Peer Review Report
- Peer Review Defect List

- Bugs Reported by Testing

Total number of defects found by Reviews**Description**

This metric will indicate the total number of defects identified by the Review process. The defects are further categorized as High, Medium or Low

Formula

Total number of defects identified in the Project

Metric Representation

- Defects per Stage

Calculated at

- Completion of Reviews

Calculated from

- Peer Review Report
- Peer Review Defect List

Defects vs. Review effort – Review Yield**Description**

This metric will indicate the effort expended in each stage for reviews to the defects found

Formula

- Defects / Review effort

Metric Representation

- Defects / Review effort

Calculated at

- Completion of Reviews

Calculated from

- Peer Review Report
- Peer Review Defect List

Requirements Stability Index (RSI)**Description**

This metric gives the stability factor of the requirements over a period of time, after the requirements have been mutually agreed and baselined between Ivesia Solutions and the Client

Formula

- $RSI = 100 * [(\text{Number of baselined requirements}) - (\text{Number of changes in requirements after the requirements are baselined})] / (\text{Number of baselined requirements})$

Metric Representation

- Percentage

Calculated at

- Stage completion and Project completion

Calculated from

- Change Request
- Software Requirements Specification

Change Requests by State**Description**

This metric provides the analysis on state of the requirements

Formula

- Number of accepted requirements
- Number of rejected requirements
- Number of postponed requirements

Metric Representation

- Number

Calculated at

- Stage completion

Calculated from

- Change Request
- Software Requirements Specification

Requirements to Design Traceability**Description**

This metric provides the analysis on the number of requirements designed to the number of requirements that were not designed

Formula

- Total Number of Requirements
- Number of Requirements Designed
- Number of Requirements not Designed

Metric Representation

- Number

Calculated at

- Stage completion

Calculated from

- SRS
- Detail Design

Design to Requirements Traceability**Description**

This metric provides the analysis on the number of design elements matching requirements to the number of design elements not matching requirements

Formula

- Number of Design elements
- Number of Design elements matching Requirements
- Number of Design elements not matching Requirements

Metric Representation

- Number

Calculated at

- Stage completion

Calculated from

- SRS
- Detail Design

Requirements to Test case Traceability**Description**

This metric provides the analysis on the number of requirements tested Vs the number of requirements not tested

Formula

- Number of Requirements
- Number of Requirements Tested
- Number of Requirements not Tested

Metric Representation

- Number

Calculated at

- Stage completion

Calculated from

- SRS
- Detail Design

- Test Case Specification

Test cases to Requirements traceability**Description**

This metric provides the analysis on the number of test cases matching requirements Vs the number of test cases not matching requirements

Formula

- Number of Requirements
- Number of Test cases with matching Requirements
- Number of Test cases not matching Requirements

Metric Representation

- Number

Calculated at

- Stage completion

Calculated from

- SRS
- Test Case Specification

Number of defects in coding found during testing by severity**Description**

This metric provides the analysis on the number of defects by the severity

Formula

- Number of Defects
- Number of defects of low priority
- Number of defects of medium priority
- Number of defects of high priority

Metric Representation

- Number

Calculated at

- Stage completion

Calculated from

- Bug Report

Defects – Stage of origin, detection, removal**Description**

This metric provides the analysis on the number of defects by the stage of origin, detection and removal.

Formula

- Number of Defects
- Stage of origin
- Stage of detection
- Stage of removal

Metric Representation

- Number

Calculated at

- Stage completion

Calculated from

- Bug Report

Defect Density**Description**

This metric provides the analysis on the number of defects to the size of the work product

Formula

Defect Density = [Total no. of Defects / Size (FP / KLOC)] * 100

Metric Representation

- Percentage

Calculated at

- Stage completion

Calculated from

- Defects List
- Bug Report

How do you determine metrics for your application?

Objectives of Metrics are not only to measure but also understand the progress to the Organizational Goal.

The Parameters for determining the Metrics for an application:

- Duration
- Complexity
- Technology Constraints
- Previous Experience in Same Technology
- Business Domain
- Clarity of the scope of the project

One interesting and useful approach to arrive at the suitable metrics is using the Goal-Question-Metric Technique.

As evident from the name, the GQM model consists of three layers; a Goal, a Set of Questions, and lastly a Set of Corresponding Metrics. It is thus a hierarchical structure starting with a goal (specifying purpose of measurement, object to be measured, issue to be measured, and viewpoint from which the measure is taken). The goal is refined into several questions that usually break down the issue into its major components. Each question is then refined into metrics, some of them objective, some of them subjective. The same metric can be used in order to answer different questions under the same goal. Several GQM models can also have questions and metrics in common, making sure that, when the measure is actually taken, the different viewpoints are taken into account correctly (i.e., the metric might have different values when taken from different viewpoints).

In order to give an example of application of the model:

Goal Purpose Issue Object View Point	Improve the timeliness of Change Request Processing from the Project Manager's viewpoint
Question	What is the current Change Request Processing Speed?
Metric	Average Cycle Time Standard Deviation % cases outside of the upper limit
Question	Is the performance of the process improving?
Metric	Current average cycle time Baseline average cycle time 100 * Subjective rating of manager's satisfaction

When do you determine Metrics?

When the requirements are understood in a high-level, at this stage, the team size, project size must be known to an extent, in which the project is at a "defined" stage.

References

- Effective Methods of Software Testing, William E Perry.
- Software Engineering – A Practitioners Approach, Roger Pressman.
- An API Testing Method by Alan A Jorgensen and James A Whittaker.
- API Testing Methodology by Anoop Kumar P, working for Novell Software Development (I) Pvt Ltd., Bangalore.
- “Why is API Testing Different “by Nikhil Nilakantan, Hewlett Packard and Ibrahim K. El-Far, Florida Institute of Technology.
- Test Strategy & Test Plan Preparation – Training course attended @ SoftSmith
- Designing Test Cases - Cem Kaner, J.D., Ph.D.
- Scenario Testing - Cem Kaner, J.D., Ph.D.
- Exploratory Testing Explained, v.1.3 4/16/03 by James Bach.
- Exploring Exploratory Testing by Andy Tinkham and Cem Kaner.
- Session-Based Test Management by Jonathan Bach (first published in Software Testing and Quality Engineering magazine, 11/00).
- Defect Driven Exploratory Testing (DDET) by Ananthalakshmi.
- Software Engineering Body of Knowledge v1.0
(<http://www.sei.cmu.edu/publications>)
- Unit Testing guidelines by Scott Highet (<http://www.Stickyminds.com>)
- <http://www.sasystems.com>
- <http://www.softwareqatest.com>
- http://www.eng.mu.edu/corlissg/198.2001/KFN_ch11-tools.html
- <http://www.ics.uci.edu/~jrobbins/ics125w04/nonav/howto-reviews.html>
- IEEE SOFTWARE REVIEWS Std 1028-1997
- <http://www.agilemanifesto.org>
- <http://www.processimpact.com>
- The Goal Question Metric Approach, Victor R. Basili¹ Gianluigi Caldiera¹ H. Dieter Rombach²
- <http://www.webopedia.com>

GNU Free Documentation License

Version 1.2, November 2002

Copyright (C) 2000,2001,2002 Free Software Foundation, Inc.

59 Temple Place, Suite 330, Boston, MA 02111-1307 USA

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

0. PREAMBLE

The purpose of this License is to make a manual, textbook, or other functional and useful document "free" in the sense of freedom: to assure everyone the effective freedom to copy and redistribute it, with or without modifying it, either commercially or noncommercially. Secondly, this License preserves for the author and publisher a way to get credit for their work, while not being considered responsible for modifications made by others.

This License is a kind of "copyleft", which means that derivative works of the document must themselves be free in the same sense. It complements the GNU General Public License, which is a copyleft license designed for free software.

We have designed this License in order to use it for manuals for free software, because free software needs free documentation: a free program should come with manuals providing the same freedoms that the software does. But this License is not limited to software manuals; it can be used for any textual work, regardless of subject matter or whether it is published as a printed book. We recommend this License principally for works whose purpose is instruction or reference.

1. APPLICABILITY AND DEFINITIONS

This License applies to any manual or other work, in any medium, that contains a notice placed by the copyright holder saying it can be distributed under the terms of this License. Such a notice grants a world-wide, royalty-free license, unlimited in duration, to use that work under the conditions stated herein. The "Document", below, refers to any such manual or work. Any member of the public is a licensee, and is addressed as "you". You accept the license if you copy, modify or distribute the work in a way requiring permission under copyright law.

A "Modified Version" of the Document means any work containing the Document or a portion of it, either copied verbatim, or with modifications and/or translated into another language.

A "Secondary Section" is a named appendix or a front-matter section of the Document that deals exclusively with the relationship of the publishers or authors of the Document to the Document's overall subject (or to related matters) and contains nothing that could fall directly within that overall subject. (Thus, if the Document is in part a textbook of mathematics, a Secondary Section may not explain any mathematics.) The relationship could be a matter of historical connection with the subject or with related matters, or of legal, commercial, philosophical, ethical or political position regarding them.

The "Invariant Sections" are certain Secondary Sections whose titles are designated, as being those of Invariant Sections, in the notice that says that the Document is released under this License. If a section does not fit the above definition of Secondary then it is not allowed to be designated as Invariant. The Document may contain zero Invariant Sections. If the Document does not identify any Invariant Sections then there are none.

The "Cover Texts" are certain short passages of text that are listed, as Front-Cover Texts or Back-Cover Texts, in the notice that says that the Document is released under this License. A Front-Cover Text may be at most 5 words, and a Back-Cover Text may be at most 25 words.

A "Transparent" copy of the Document means a machine-readable copy, represented in a format whose specification is available to the general public, that is suitable for revising the document straightforwardly with generic text editors or (for images composed of pixels) generic paint programs or (for drawings) some widely available drawing editor, and that is suitable for input to text formatters or for automatic

translation to a variety of formats suitable for input to text formatters. A copy made in an otherwise Transparent file format whose markup, or absence of markup, has been arranged to thwart or discourage subsequent modification by readers is not Transparent. An image format is not Transparent if used for any substantial amount of text. A copy that is not "Transparent" is called "Opaque".

Examples of suitable formats for Transparent copies include plain ASCII without markup, Texinfo input format, LaTeX input format, SGML or XML using a publicly available DTD, and standard-conforming simple HTML, PostScript or PDF designed for human modification. Examples of transparent image formats include PNG, XCF and JPG. Opaque formats include proprietary formats that can be read and edited only by proprietary word processors, SGML or XML for which the DTD and/or processing tools are not generally available, and the machine-generated HTML, PostScript or PDF produced by some word processors for output purposes only.

The "Title Page" means, for a printed book, the title page itself, plus such following pages as are needed to hold, legibly, the material this License requires to appear in the title page. For works in formats which do not have any title page as such, "Title Page" means the text near the most prominent appearance of the work's title, preceding the beginning of the body of the text.

A section "Entitled XYZ" means a named subunit of the Document whose title either is precisely XYZ or contains XYZ in parentheses following text that translates XYZ in another language. (Here XYZ stands for a specific section name mentioned below, such as "Acknowledgements", "Dedications", "Endorsements", or "History".) To "Preserve the Title" of such a section when you modify the Document means that it remains a section "Entitled XYZ" according to this definition.

The Document may include Warranty Disclaimers next to the notice which states that this License applies to the Document. These Warranty Disclaimers are considered to be included by reference in this License, but only as regards disclaiming warranties: any other implication that these Warranty Disclaimers may have is void and has no effect on the meaning of this License.

2. VERBATIM COPYING

You may copy and distribute the Document in any medium, either commercially or noncommercially, provided that this License, the copyright notices, and the license notice saying this License applies to the Document are reproduced in all copies, and that you add no other conditions whatsoever to those of this License. You may not use technical measures to obstruct or control the reading or further copying of the copies you make or distribute. However, you may accept compensation in exchange for copies. If you distribute a large enough number of copies you must also follow the conditions in section 3.

You may also lend copies, under the same conditions stated above, and you may publicly display copies.

3. COPYING IN QUANTITY

If you publish printed copies (or copies in media that commonly have printed covers) of the Document, numbering more than 100, and the Document's license notice requires Cover Texts, you must enclose the copies in covers that carry, clearly and legibly, all these Cover Texts: Front-Cover Texts on the front cover, and Back-Cover Texts on the back cover. Both covers must also clearly and legibly identify you as the publisher of these copies. The front cover must present the full title with all words of the title equally prominent and visible. You may add other material on the covers in addition. Copying with changes limited to the covers, as long as they preserve the title of the Document and satisfy these conditions, can be treated as verbatim copying in other respects.

If the required texts for either cover are too voluminous to fit legibly, you should put the first ones listed (as many as fit reasonably) on the actual cover, and continue the rest onto adjacent pages.

If you publish or distribute Opaque copies of the Document numbering more than 100, you must either include a machine-readable Transparent copy along with each Opaque

copy, or state in or with each Opaque copy a computer-network location from which the general network-using public has access to download using public-standard network protocols a complete Transparent copy of the Document, free of added material. If you use the latter option, you must take reasonably prudent steps, when you begin distribution of Opaque copies in quantity, to ensure that this Transparent copy will remain thus accessible at the stated location until at least one year after the last time you distribute an Opaque copy (directly or through your agents or retailers) of that edition to the public.

It is requested, but not required, that you contact the authors of the Document well before redistributing any large number of copies, to give them a chance to provide you with an updated version of the Document.

4. MODIFICATIONS

You may copy and distribute a Modified Version of the Document under the conditions of sections 2 and 3 above, provided that you release the Modified Version under precisely this License, with the Modified Version filling the role of the Document, thus licensing distribution and modification of the Modified Version to whoever possesses a copy of it. In addition, you must do these things in the Modified Version:

- **A.** Use in the Title Page (and on the covers, if any) a title distinct from that of the Document, and from those of previous versions (which should, if there were any, be listed in the History section of the Document). You may use the same title as a previous version if the original publisher of that version gives permission.
- **B.** List on the Title Page, as authors, one or more persons or entities responsible for authorship of the modifications in the Modified Version, together with at least five of the principal authors of the Document (all of its principal authors, if it has fewer than five), unless they release you from this requirement.
- **C.** State on the Title page the name of the publisher of the Modified Version, as the publisher.
- **D.** Preserve all the copyright notices of the Document.
- **E.** Add an appropriate copyright notice for your modifications adjacent to the other copyright notices.
- **F.** Include, immediately after the copyright notices, a license notice giving the public permission to use the Modified Version under the terms of this License, in the form shown in the Addendum below.
- **G.** Preserve in that license notice the full lists of Invariant Sections and required Cover Texts given in the Document's license notice.
- **H.** Include an unaltered copy of this License.
- **I.** Preserve the section Entitled "History", Preserve its Title, and add to it an item stating at least the title, year, new authors, and publisher of the Modified Version as given on the Title Page. If there is no section Entitled "History" in the Document, create one stating the title, year, authors, and publisher of the Document as given on its Title Page, then add an item describing the Modified Version as stated in the previous sentence.
- **J.** Preserve the network location, if any, given in the Document for public access to a Transparent copy of the Document, and likewise the network locations given in the Document for previous versions it was based on. These may be placed in the "History" section. You may omit a network location for a work that was published at least four years before the Document itself, or if the original publisher of the version it refers to gives permission.
- **K.** For any section Entitled "Acknowledgements" or "Dedications", Preserve the Title of the section, and preserve in the section all the substance and tone of each of the contributor acknowledgements and/or dedications given therein.

- **L.** Preserve all the Invariant Sections of the Document, unaltered in their text and in their titles. Section numbers or the equivalent are not considered part of the section titles.
- **M.** Delete any section Entitled "Endorsements". Such a section may not be included in the Modified Version.
- **N.** Do not retitle any existing section to be Entitled "Endorsements" or to conflict in title with any Invariant Section.
- **O.** Preserve any Warranty Disclaimers.

If the Modified Version includes new front-matter sections or appendices that qualify as Secondary Sections and contain no material copied from the Document, you may at your option designate some or all of these sections as invariant. To do this, add their titles to the list of Invariant Sections in the Modified Version's license notice. These titles must be distinct from any other section titles.

You may add a section Entitled "Endorsements", provided it contains nothing but endorsements of your Modified Version by various parties--for example, statements of peer review or that the text has been approved by an organization as the authoritative definition of a standard.

You may add a passage of up to five words as a Front-Cover Text, and a passage of up to 25 words as a Back-Cover Text, to the end of the list of Cover Texts in the Modified Version. Only one passage of Front-Cover Text and one of Back-Cover Text may be added by (or through arrangements made by) any one entity. If the Document already includes a cover text for the same cover, previously added by you or by arrangement made by the same entity you are acting on behalf of, you may not add another; but you may replace the old one, on explicit permission from the previous publisher that added the old one.

The author(s) and publisher(s) of the Document do not by this License give permission to use their names for publicity for or to assert or imply endorsement of any Modified Version.

5. COMBINING DOCUMENTS

You may combine the Document with other documents released under this License, under the terms defined in section 4 above for modified versions, provided that you include in the combination all of the Invariant Sections of all of the original documents, unmodified, and list them all as Invariant Sections of your combined work in its license notice, and that you preserve all their Warranty Disclaimers.

The combined work need only contain one copy of this License, and multiple identical Invariant Sections may be replaced with a single copy. If there are multiple Invariant Sections with the same name but different contents, make the title of each such section unique by adding at the end of it, in parentheses, the name of the original author or publisher of that section if known, or else a unique number. Make the same adjustment to the section titles in the list of Invariant Sections in the license notice of the combined work.

In the combination, you must combine any sections Entitled "History" in the various original documents, forming one section Entitled "History"; likewise combine any sections Entitled "Acknowledgements", and any sections Entitled "Dedications". You must delete all sections Entitled "Endorsements."

6. COLLECTIONS OF DOCUMENTS

You may make a collection consisting of the Document and other documents released under this License, and replace the individual copies of this License in the various documents with a single copy that is included in the collection, provided that you follow the rules of this License for verbatim copying of each of the documents in all other respects.

You may extract a single document from such a collection, and distribute it individually under this License, provided you insert a copy of this License into the extracted

document, and follow this License in all other respects regarding verbatim copying of that document.

7. AGGREGATION WITH INDEPENDENT WORKS

A compilation of the Document or its derivatives with other separate and independent documents or works, in or on a volume of a storage or distribution medium, is called an "aggregate" if the copyright resulting from the compilation is not used to limit the legal rights of the compilation's users beyond what the individual works permit. When the Document is included in an aggregate, this License does not apply to the other works in the aggregate which are not themselves derivative works of the Document.

If the Cover Text requirement of section 3 is applicable to these copies of the Document, then if the Document is less than one half of the entire aggregate, the Document's Cover Texts may be placed on covers that bracket the Document within the aggregate, or the electronic equivalent of covers if the Document is in electronic form. Otherwise they must appear on printed covers that bracket the whole aggregate.

8. TRANSLATION

Translation is considered a kind of modification, so you may distribute translations of the Document under the terms of section 4. Replacing Invariant Sections with translations requires special permission from their copyright holders, but you may include translations of some or all Invariant Sections in addition to the original versions of these Invariant Sections. You may include a translation of this License, and all the license notices in the Document, and any Warranty Disclaimers, provided that you also include the original English version of this License and the original versions of those notices and disclaimers. In case of a disagreement between the translation and the original version of this License or a notice or disclaimer, the original version will prevail. If a section in the Document is Entitled "Acknowledgements", "Dedications", or "History", the requirement (section 4) to Preserve its Title (section 1) will typically require changing the actual title.

9. TERMINATION

You may not copy, modify, sublicense, or distribute the Document except as expressly provided for under this License. Any other attempt to copy, modify, sublicense or distribute the Document is void, and will automatically terminate your rights under this License. However, parties who have received copies, or rights, from you under this License will not have their licenses terminated so long as such parties remain in full compliance.

10. FUTURE REVISIONS OF THIS LICENSE

The Free Software Foundation may publish new, revised versions of the GNU Free Documentation License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns. See <http://www.gnu.org/copyleft/>.

Each version of the License is given a distinguishing version number. If the Document specifies that a particular numbered version of this License "or any later version" applies to it, you have the option of following the terms and conditions either of that specified version or of any later version that has been published (not as a draft) by the Free Software Foundation. If the Document does not specify a version number of this License, you may choose any version ever published (not as a draft) by the Free Software Foundation.